

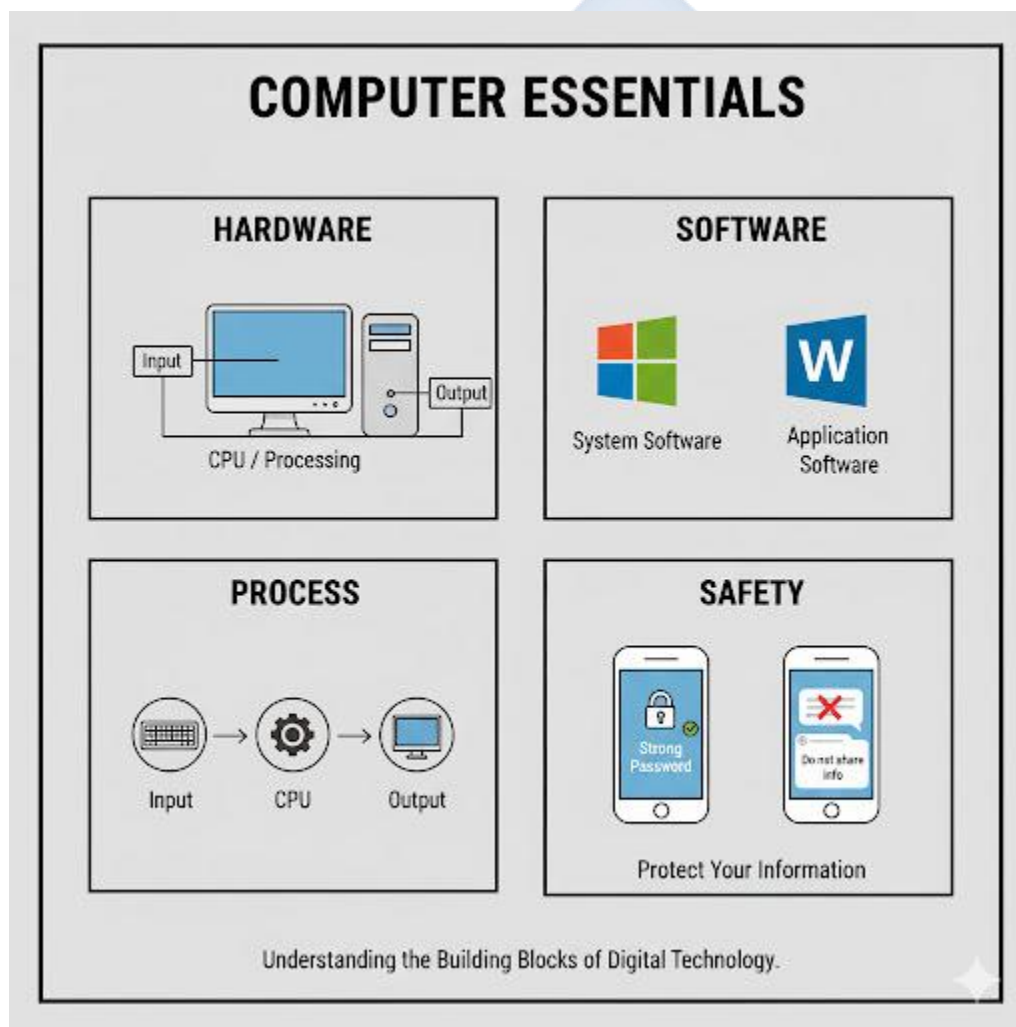
FIRST TERM – JSS 2 DIGITAL TECHNOLOGIES LESSON NOTES

WEEK 1: REVIEW OF JSS1 WORK

Topic: Review of JSS1 Work

Breakdown (Subtopics):

- Recap of hardware, software, and online safety



Performance Objectives: By the end of this lesson, students should be able to:

1. Define a computer and its two main components: hardware and software.
2. List and categorize at least two examples of hardware devices (input, output, processing, storage).
3. Differentiate between system software and application software with examples.
4. Define "Online Safety" (or "Cybersecurity").
5. List at least five essential rules for staying safe online.

Lesson Content:

1. Recap: What is a Computer? A computer is an electronic device that can **accept data** (input), **process** it according to stored instructions, **store** the results, and **produce information** (output).

2. Recap: Hardware

- **Definition:** Hardware refers to all the **physical parts** of a computer system that you can see and touch.
- **Categories and Examples:**
 - **Input Devices:** Used to enter data *into* the computer.
 - *Examples:* Keyboard, mouse, scanner, microphone, webcam.
 - **Processing Devices:** The "brain" of the computer where all calculations and instructions are carried out.
 - *Examples:* **CPU (Central Processing Unit)**, **RAM (Random Access Memory)** (for temporary processing).
 - **Output Devices:** Used to present processed information (output) *from* the computer.
 - *Examples:* Monitor (screen), printer, speakers, projector.
 - **Storage Devices:** Used to store data and programs permanently (or semi-permanently).
 - *Examples:* Hard Disk Drive (HDD), Solid State Drive (SSD), USB Flash Drive, CD/DVD, Memory Card.

3. Recap: Software

- **Definition:** Software refers to the set of **instructions, programs, and data** that tells the hardware *what* to do and *how* to do it. You cannot physically touch software.
- **Types of Software:**
 - **A. System Software:**
 - **Function:** Manages and controls the computer's hardware. It is the platform for all other software.
 - **Examples:**
 - **Operating Systems (OS):** Microsoft Windows 11, Apple macOS, Linux, Android, iOS.
 - **Utilities:** Antivirus software, disk cleaners.
 - **B. Application Software:**
 - **Function:** Programs designed to perform *specific tasks* for the user.
 - **Examples:**
 - **Word Processors:** Microsoft Word, Google Docs.
 - **Web Browsers:** Google Chrome, Firefox.
 - **Games:** FIFA, Minecraft.
 - **Design:** CorelDraw, Adobe Photoshop.

4. Recap: Online Safety (Cybersecurity)

- **Definition:** Online safety (or cybersecurity) is the practice of protecting oneself and one's information from threats and dangers on the internet.
- **Importance:** The internet is a public space, and just like in a real city, there are dangers like thieves (hackers), bullies, and scammers.
- **Essential Safety Rules:**
 1. **Protect Personal Information:** NEVER share your full name, home address, phone number, school name, or parents' bank details online.
 2. **Use Strong Passwords:** Create long, complex passwords (using letters, numbers, and symbols). Do not use "12345" or "password".
 3. **Do Not Talk to Strangers:** Do not accept friend requests or reply to messages from people you do not know in real life.

4. **Beware of "Phishing":** Do not click on suspicious links or download attachments from unknown senders. They can contain viruses.
5. **Report Cyberbullying:** If someone is harassing or threatening you online, do not reply. **Block** them, **save** the evidence (screenshots), and **tell** a trusted adult (parent or teacher) immediately.

Evaluation:

1. What is the difference between hardware and software?
2. List one input device, one output device, and one storage device.
3. What is the "brain" of the computer called?
4. What is the difference between System Software (like Windows) and Application Software (like MS Word)?
5. What should you do if a stranger sends you a message online?

Assignment:

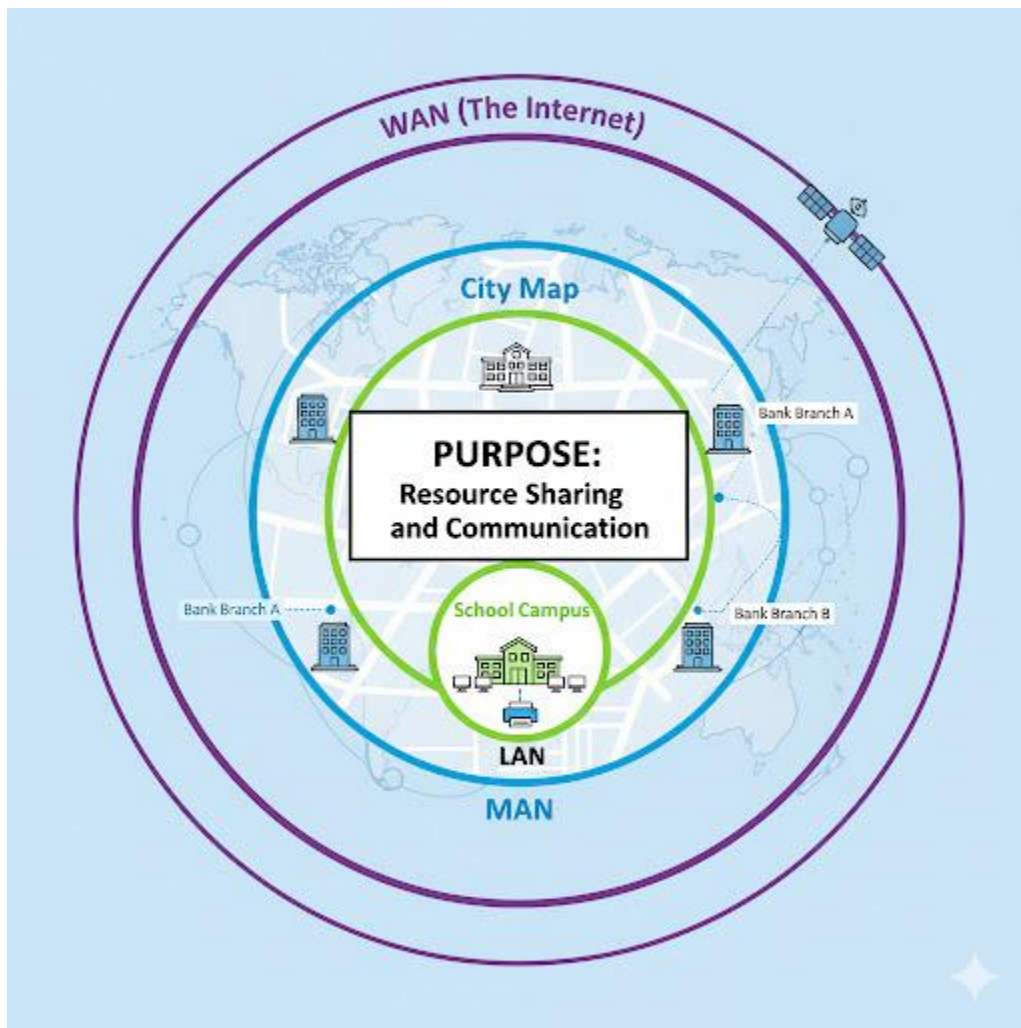
1. Define a computer.
2. Draw a simple diagram showing the relationship between Input, Processing, Output, and Storage.
3. What is an Operating System (OS)? Give two examples.
4. What is a "strong password"? Give an example of a strong password and a weak password.
5. What is "cyberbullying," and what are the three steps (Block, Save, Tell) to handle it?

WEEK 2: BASIC COMPUTER COMMUNICATION NETWORKS

Topic: Basic Computer Communication Networks

Breakdown (Subtopics):

- Meaning and need for networks
- Network types: LAN, MAN, WAN



Performance Objectives: By the end of this lesson, students should be able to:

1. Define "Computer Network."
2. State at least three reasons why we need networks (advantages).
3. Define LAN, MAN, and WAN.

4. Differentiate between the three network types based on their geographical size.
5. Give a real-world example of each network type.

Lesson Content:

1. Meaning of a Computer Network

- **Definition:** A **computer network** is a group of two or more computers and other devices (like printers or servers) that are **interconnected** (linked together) for the purpose of communicating and **sharing resources**.
- **Standalone Computer:** A computer that is *not* connected to a network is called a standalone computer.

2. Need for Networks (Advantages) Why do we bother connecting computers?

1. **Resource Sharing:** This is the main reason.
 - *Hardware Sharing:* A whole office (e.g., 20 computers) can share **one printer**, one scanner, or one large hard drive. This saves a lot of money.
 - *Internet Connection Sharing:* One internet connection can be shared among all users on the network.
2. **File and Data Sharing:** Users can easily access and share the same files (like a company database or a school project) from different computers.
3. **Communication:** Networks allow for fast, cheap, and easy communication.
 - *Examples:* Email, instant messaging (like WhatsApp), video conferencing (like Zoom).
4. **Centralized Management:** It is easier to manage, secure, and back up data that is stored in one central location (a server) rather than on 50 different computers.

3. Types of Networks (Based on Geographical Size) Networks are classified by how much area they cover.

- **A. LAN (Local Area Network)**
 - **Definition:** A network that covers a **small geographical area**.

- **Scope:** Typically confined to a single room, a single building, or a small group of buildings (like a school campus or a hospital).
- **Example:** The computers in your school's computer lab, or the computers and printer in your home, all connected to one Wi-Fi router.
- **B. MAN (Metropolitan Area Network)**
 - **Definition:** A high-speed network that covers a **medium-sized geographical area**, like an entire city or a large town.
 - **Scope:** It is larger than a LAN but smaller than a WAN. It often connects several LANs together.
 - **Example:** A bank connecting all its branches within Lagos city. A cable TV network covering a whole city.
- **C. WAN (Wide Area Network)**
 - **Definition:** A network that covers a **very large geographical area**, such as a state, a country, or even the whole world.
 - **Scope:** A WAN is a collection of interconnected LANs or MANs, often connected using telephone lines, satellites, or fibre optic cables.
 - **Example:** The **Internet** is the largest WAN in the world. A company like MTN or a large bank (like UBA) connecting all its branches *across Nigeria* forms a WAN.

Evaluation:

1. What is a computer network?
2. List three main reasons for having a computer network.
3. What do the acronyms LAN, MAN, and WAN stand for?
4. What is the main difference between a LAN and a WAN?
5. What is the largest example of a WAN?

Assignment:

1. Define "Computer Network" in your own words.

2. Explain the concept of "resource sharing" and give two examples (one hardware, one software/data).
3. Create a small table comparing LAN, MAN, and WAN. Include columns for "Full Name," "Size," and "Example."
4. What type of network (LAN, MAN, or WAN) would your school's computer lab be?
5. What type of network would be used to connect all the branches of First Bank across Africa?

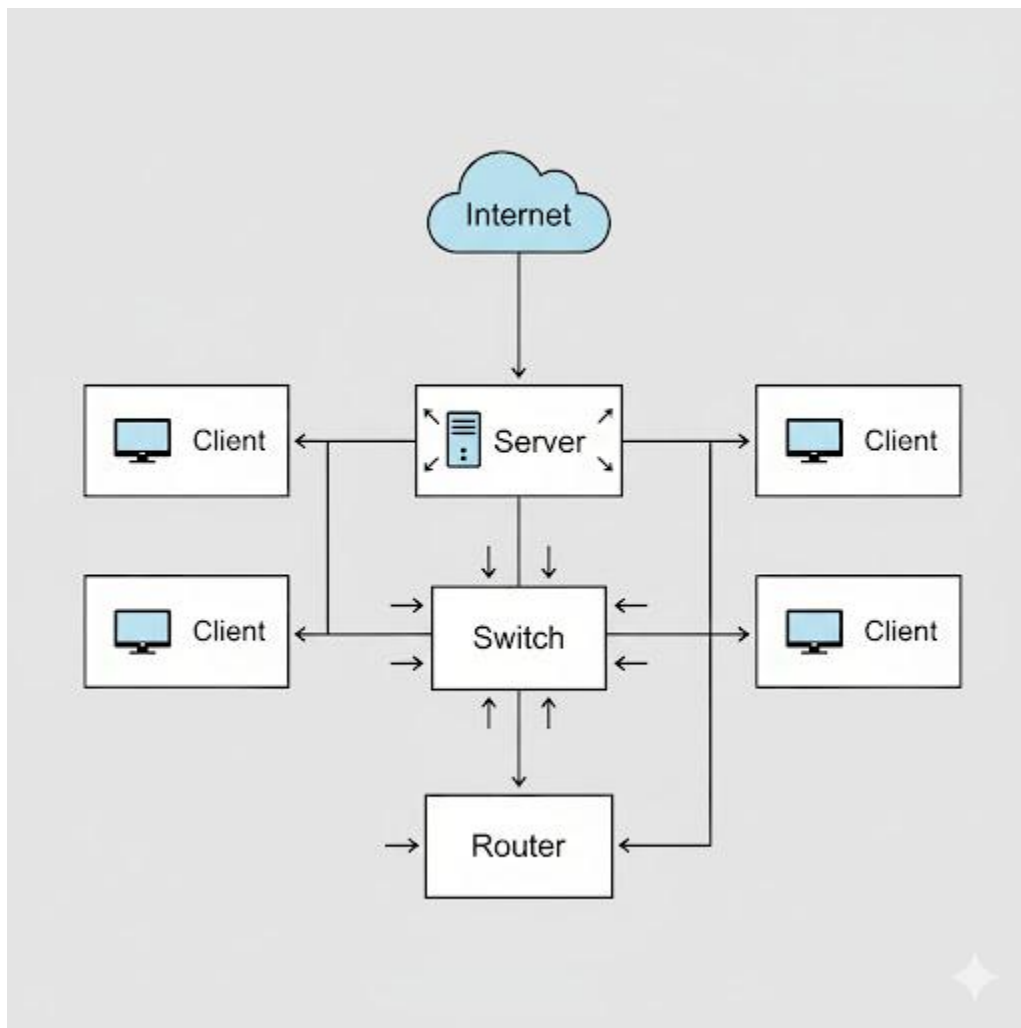


WEEK 3: COMPONENTS OF A COMPUTER NETWORK

Topic: Components of a Computer Network

Breakdown (Subtopics):

- Servers, clients, switches, routers, hubs
- Network media: cables, wireless



Performance Objectives: By the end of this lesson, students should be able to:

1. Define "Server" and "Client" and explain their relationship.

2. Differentiate between a Hub and a Switch.
3. State the primary function of a Router.
4. Define "Network Media" and list the two main categories (cabled and wireless).
5. Identify a Wi-Fi signal as a form of wireless media.

Lesson Content:

To build a network (like a LAN), you need several key components:

1. Core Components (The Computers)

- **Server:**
 - **Definition:** A server is a very powerful, high-capacity computer whose purpose is to **"serve" (provide) resources** to other computers on the network.
 - **Function:** It can be a *file server* (storing all the files), a *print server* (managing the printer), or a *web server* (hosting a website).
- **Client (or Node/Workstation):**
 - **Definition:** A client is a regular computer (like your desktop, laptop, or smartphone) that **requests and uses** the resources provided by the server.
 - **Relationship:** In a "Client-Server" network, many clients are connected to one central server.

2. Network Hardware (The Connection Devices)

- **Network Interface Card (NIC):**
 - **Function:** A piece of hardware *inside* the computer that allows it to physically connect to the network. It provides the "port" for the network cable (or the antenna for Wi-Fi).
- **Hub:**
 - **Function:** A simple, older device used to connect multiple computers in a LAN (in a Star topology).

- **How it works (Inefficient):** When a hub receives data, it "shouts" and broadcasts that data to *every* computer connected to it, whether they need it or not. This creates unnecessary traffic.
- **Switch:**
 - **Function:** A modern, "intelligent" device used to connect multiple computers in a LAN.
 - **How it works (Efficient):** A switch is *smart*. It learns the address of each computer. When it receives data, it sends it *only* to the specific computer that the data is intended for. This is much faster and more secure than a hub.
- **Router:**
 - **Function:** This is a "gateway" device. Its main job is to **connect different networks together**.
 - **Example:** Your Wi-Fi router at home is a router. Its job is to connect your *home LAN* (your phone, your laptop) to the *Internet WAN* (the outside world). It "routes" traffic between the two networks.

3. Network Media (The Path) Network media is the physical *pathway* through which data travels.

- **A. Cabled Media (Wired):**
 - **Twisted Pair (Ethernet Cable):** The most common cable for LANs. It looks like a telephone cord but is thicker.
 - **Fibre Optic Cable:** Transmits data as pulses of *light* down a thin glass strand. It is *extremely fast*, expensive, and used for high-speed connections (like WANs or MANs).
- **B. Wireless Media:**
 - **Function:** Transmits data using electromagnetic waves (like radio waves) through the air, without needing cables.
 - **Examples:**
 - **Wi-Fi (Wireless Fidelity):** Uses radio waves to create a wireless LAN (WLAN). This is what we use in homes, schools, and "hotspots."

- **Bluetooth:** A short-range wireless technology for connecting devices (e.g., phone to earpiece).

Evaluation:

1. What is the difference between a server and a client?
2. What is the main job of a router?
3. What is a "smarter" and more efficient device: a Hub or a Switch? Why?
4. What is "network media"?
5. List the two main categories of network media and give one example of each.

Assignment:

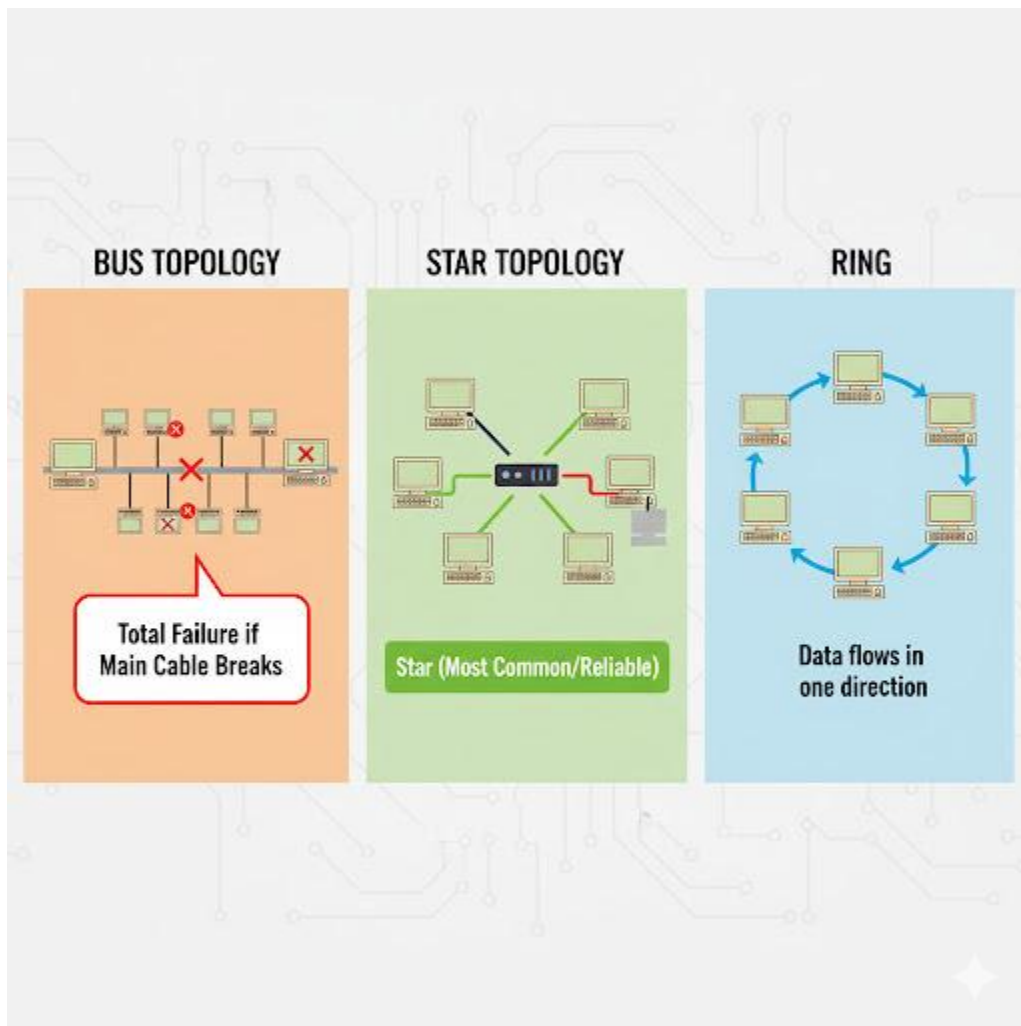
1. Define Server, Client, Switch, and Router.
2. What is the full meaning of NIC, and what is its function?
3. What is the difference between a Hub and a Switch?
4. What is the difference between an Ethernet cable and Wi-Fi?
5. Look at the "box" that gives you internet at home. Is it a Hub, Switch, or Router? (Hint: It connects you to the internet).

WEEK 4: NETWORK TOPOLOGIES

Topic: Network Topologies

Breakdown (Subtopics):

- Meaning and types: bus, star, ring
- Advantages and disadvantages



Performance Objectives: By the end of this lesson, students should be able to:

1. Define "Network Topology."
2. List the three main types of network topologies.
3. Draw and label a simple diagram for a Bus, Star, and Ring topology.

4. State one major advantage for each topology.
5. State one major disadvantage for each topology.

Lesson Content:

1. Meaning of Network Topology

- **Definition:** A **network topology** is the **physical or logical layout** (the arrangement or "map") of a network. It describes how all the computers (nodes) and devices are physically connected to each other.

2. Types of Topologies

- **A. Bus Topology**

- **Description:** This is the simplest topology. All computers and devices are connected one by one to a **single main cable**, called the "**backbone**" or "**bus**".
- **How it works:** Data is sent onto the main cable. All computers "listen" for data addressed to them. "Terminators" are placed at each end of the cable to stop the signal from bouncing.
- **Advantages:**
 1. Very simple and easy to install.
 2. Uses the least amount of cable, making it cheap.
- **Disadvantages:**
 1. **Main Cable Failure:** If the main backbone cable breaks *anywhere*, the **entire network fails**.
 2. **Difficult to Troubleshoot:** It is hard to find the *exact* location of a problem.
 3. **Low Security:** All computers can "see" all the data.

- **B. Star Topology**

- **Description:** This is the most common topology used in modern LANs. All computers are connected *individually* to a **central device** (a **Switch** or a **Hub**). It looks like a star.

- **How it works:** All data must pass *through* the central switch. The switch (if it's a smart switch) then directs the data *only* to the intended computer.
- **Advantages:**
 1. **Reliable (Fault-Tolerant):** If one computer's cable breaks, only *that one computer* is affected. The rest of the network continues to work.
 2. **Easy to Manage:** Easy to add new computers or find faults (the problem is either the computer or its cable).
- **Disadvantages:**
 1. **Central Point of Failure:** If the central **switch or hub fails**, the **entire network fails**.
 2. **More Cable:** Uses more cable than a bus topology, so it can be more expensive.
- **C. Ring Topology**
 - **Description:** In this topology, each computer is connected directly to *two other computers*, one on each side, forming a **continuous circle or ring**.
 - **How it works:** Data travels around the ring in *one direction*, passing from one computer to the next until it reaches its destination.
 - **Advantages:**
 1. Works well under heavy traffic as there are no "collisions" (data only moves one way).
 2. No central server or switch is needed.
 - **Disadvantages:**
 1. **Single Point of Failure:** If *any computer* or *any cable* in the ring breaks, the **entire network loop is broken** and fails.
 2. Difficult to add or remove computers (you must "break" the ring to do so).

Evaluation:

1. What is a network topology?
2. What is the name of the topology that uses a single main backbone cable?
3. What is the name of the topology that uses a central switch or hub?
4. What is the main *disadvantage* of a Bus topology?

5. What is the main *advantage* of a Star topology?

Assignment:

1. Define topology. List the three types: Bus, Star, and Ring.
2. Draw a simple, labelled diagram for a **Bus** topology.
3. Draw a simple, labelled diagram for a **Star** topology.
4. Draw a simple, labelled diagram for a **Ring** topology.
5. Which topology is most common in schools and offices today, and why?

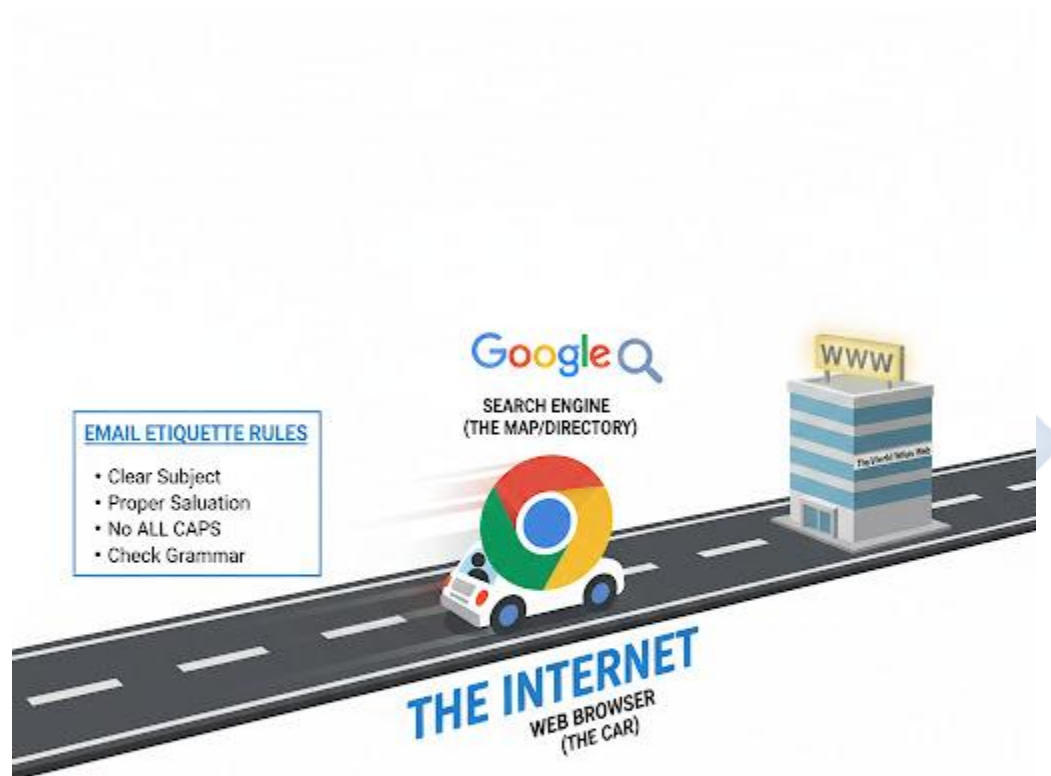


WEEK 6: INTERNET AND WEB SERVICES

Topic: Internet and Web Services

Breakdown (Subtopics):

- Search engines and browsers
- Email setup and etiquette



Performance Objectives: By the end of this lesson, students should be able to:

1. Define the "Internet" and the "World Wide Web" (WWW).

2. Differentiate between a Web Browser and a Search Engine.
3. List two examples of browsers and two examples of search engines.
4. Define "Email" and list the components of an email address.
5. List five rules of "email etiquette."

Lesson Content:

1. Internet and World Wide Web (WWW)

- **The Internet:** (Review) The global **network of networks** (a WAN). It is the *hardware* (cables, satellites, routers) that connects the world.
- **The World Wide Web (WWW):** This is a *service* that runs *on* the internet. It is a vast collection of interconnected **websites and webpages** that you can access.
- **Analogy:** The Internet is the *physical road system* of the world. The World Wide Web is all the *buildings, shops, and houses* on the side of the road.

2. Web Browsers vs. Search Engines This is a very common point of confusion.

- **A. Web Browser (The "Car")**
 - **Definition:** A **software application** (Application Software) that is installed on your computer. Its job is to *access, retrieve, and display* websites.
 - **Analogy:** The browser is the "car" or "bus" you use to *travel* on the internet roads to *visit* the shops (websites).
 - **Examples:** Google Chrome, Mozilla Firefox, Microsoft Edge, Apple Safari.
- **B. Search Engine (The "Map" or "Directory")**
 - **Definition:** A special **website** (not software on your computer) that helps you *find* other websites. You use it by typing in keywords.
 - **Analogy:** The search engine is the "map" or "directory" that tells you the *address* of the shop you are looking for.
 - **Examples:** <https://www.google.com/search?q=Google.com>, Bing.com, Yahoo.com.

3. Email (Electronic Mail)

- **Definition:** Email is a service that allows users to send and receive digital messages (with text, photos, and files) over the internet.
- **Email Address Structure:**
 - username @ domain.com
 - **username:** Your unique name (e.g., jss2student).
 - **@ Symbol:** The "at" sign, which separates the user from the domain.
 - **domain.com:** The name of the *email provider* (e.g., gmail.com, yahoo.com, outlook.com).

4. Email Etiquette (The Rules of Polite Email) When writing a formal email (e.g., to a teacher, a company, or for an application), you must be professional.

1. **Use a Clear Subject Line:** The subject line should summarize *exactly* what the email is about (e.g., "Assignment Submission for JSS2 Digital Tech" or "Question About Week 6 Lesson").
2. **Use a Proper Salutation:** Start with "Dear Sir," "Dear Ma," or "Dear Mr. Adebayo."
3. **Be Clear and Concise:** Write clearly. Do not use "text message" abbreviations (like "u," "r," "lol," "b4").
4. **Use a Proper Closing:** End with "Yours sincerely," "Yours faithfully," or "Best regards," followed by your full name.
5. **Check Spelling and Grammar:** Re-read your email to check for mistakes before sending.
6. **Do NOT Use ALL CAPS:** WRITING IN ALL CAPS IS THE SAME AS SHOUTING.

Evaluation:

1. What is the difference between the Internet and the World Wide Web?
2. What is the difference between a Web Browser and a Search Engine?
3. Is "Google Chrome" a browser or a search engine?
4. Is "https://www.google.com/search?q=Google.com" a browser or a search engine?
5. What are two important rules of email etiquette?

Assignment:

1. Define Internet, WWW, Web Browser, and Search Engine.
2. Give two examples of browsers and two examples of search engines.
3. Label the parts of this email address: student.name@school.org. (username, @, domain).
4. What is "email etiquette"?
5. Write a short, formal email to your teacher (a fictional one: teacher@school.com) asking a question about the homework. Follow all the rules of etiquette.



WEEK 8: CLOUD COMPUTING

Topic: Cloud Computing

Breakdown (Subtopics):

- Meaning of cloud storage
- Examples: Google Drive, Dropbox
- Advantages and safety



Performance Objectives: By the end of this lesson, students should be able to:

1. Define "Cloud Computing" and "Cloud Storage."
2. Differentiate between local storage and cloud storage.
3. List at least two examples of cloud storage providers.
4. State three advantages of using cloud storage.
5. State two disadvantages (risks) of using cloud storage.

Lesson Content:

1. Meaning of Cloud Computing

- **Local Storage:** This is the storage *on your device* (e.g., your phone's memory, your computer's hard drive). It is local to you.
- **Cloud Computing:** This is a broad term for using a network of **remote servers** (computers in a giant warehouse, or "data centre") hosted on the **Internet** to store, manage, and process data, instead of using your local computer.
- **"The Cloud"** is just a friendly name for the **Internet and these remote servers**.

2. Cloud Storage

- **Definition:** Cloud storage is the most common form of cloud computing. It is the service of **storing your digital files** (photos, documents, videos, backups) on these secure remote servers.
- **How it works:** You "upload" your file to the service. The service (e.g., Google) saves it in its data centre. You can then "download" or access that file from any other device (your phone, another computer) just by logging into your account.

3. Examples of Cloud Storage Providers

- **Google Drive:** (Provides 15GB free, linked to your Gmail account).
- **Dropbox:** (A popular file-hosting service).
- **Microsoft OneDrive:** (Integrated with Windows and Microsoft Office).
- **Apple iCloud:** (Used for backing up iPhones and iPads).

4. Advantages of Cloud Storage

1. **Accessibility:** You can access your files from *any device, anywhere in the world*, as long as you have an internet connection.
2. **Backup and Safety:** It is a secure backup. If your phone is stolen or your computer breaks, your files are **not lost**. They are safe in the cloud.
3. **Collaboration:** You can easily share large files or folders with other people (e.g., for a group project) without needing a flash drive. Multiple people can even edit the same document (like Google Docs) at the same time.
4. **Saves Local Space:** It frees up space on your phone or computer.

5. Disadvantages (Risks) and Safety

1. **Requires Internet:** You **must** have an internet connection to access your files. No internet, no files.
 2. **Security Risks:** The cloud provider (e.g., Google, Dropbox) could be hacked, and your data could be stolen by criminals.
 3. **Privacy Concerns:** The company that owns the server (e.g., Google) technically has access to your files.
- **Safety Tip:** Just like any online account, you *must* protect your cloud storage account with a **very strong password** and **Two-Factor Authentication (2FA)** if possible.

Evaluation:

1. What is "The Cloud"?
2. What is "Cloud Storage"?
3. What is "Local Storage"?
4. List two well-known examples of cloud storage services.
5. State two major advantages of using cloud storage.

Assignment:

1. Define Cloud Computing and Cloud Storage.
2. Explain the main difference between local storage and cloud storage.

3. List three advantages of using cloud storage.
4. List two disadvantages (risks) of using cloud storage.
5. If your laptop crashed right now, would your schoolwork be safe? Explain how cloud storage (like Google Drive) could help with this problem.

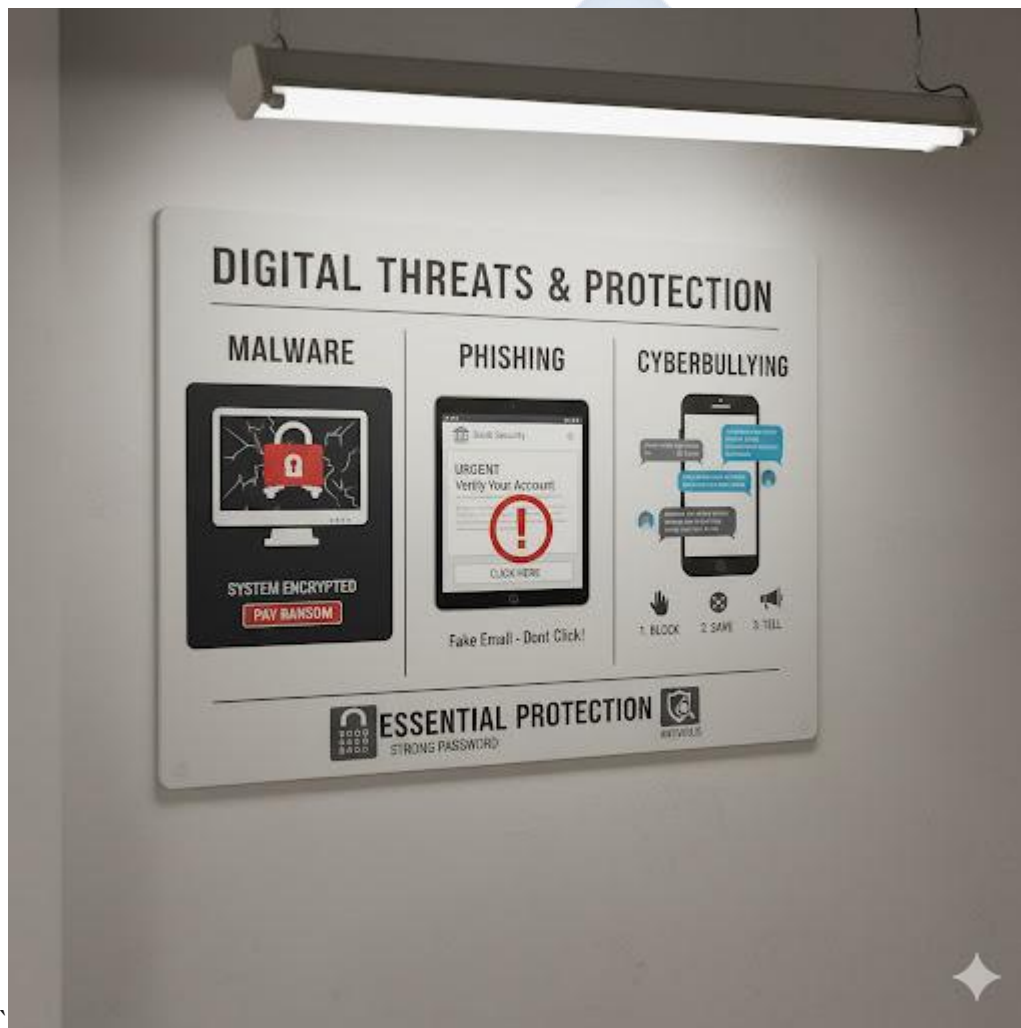


WEEK 9: CYBERSECURITY AWARENESS

Topic: Cybersecurity Awareness

Breakdown (Subtopics):

- Meaning of cybersecurity
- Types of cyberattacks
- Safe online habits



Performance Objectives: By the end of this lesson, students should be able to:

1. Define "Cybersecurity" (recap from Week 1).

2. Define "Malware," "Phishing," and "Cyberbullying."
3. Identify the signs of a phishing email.
4. List at least five safe online habits to protect against these attacks.

Lesson Content:

1. Meaning of Cybersecurity

- **Definition:** Cybersecurity is the **practice and technology** used to protect computers, networks, programs, and personal data from unauthorized access, damage, or **cyberattacks**.
- It is "Online Safety" (Week 1) but also includes the technical side (like firewalls and antivirus).

2. Types of Cyberattacks

- **A. Malware (Malicious Software):**
 - **Definition:** Software designed to damage or disrupt a computer.
 - **Virus:** A program that attaches itself to other files and spreads, corrupting data.
 - **Worm:** A standalone program that copies itself and spreads across a network.
 - **Ransomware:** A dangerous type of malware that *locks* (encrypts) all your files and demands a *ransom* (money) to unlock them.
 - **Spyware:** Secretly watches what you do on your computer and steals your passwords.
- **B. Phishing (Fishing for Information):**
 - **Definition:** An attack where a cybercriminal sends a **fake email, text message, or social media message** that *pretends* to be from a real company (like your bank, Google, or Facebook).
 - **Goal:** To *trick* you into clicking a malicious link or revealing your password, credit card number, or other personal data.
 - **How to Spot a Phishing Email:**
 1. **Sense of Urgency:** "Your account will be locked in 24 hours!"

2. **Fake Sender Address:** The email looks real, but the address is wrong (e.g., facebook.support@some-random-site.com).
3. **Generic Greeting:** "Dear Valued Customer" (Your real bank knows your name).
4. **Spelling/Grammar Mistakes.**

- **C. Cyberbullying:**

- **Definition:** (Review) Using digital technology (social media, messaging) to harass, threaten, embarrass, or target another person.
- **Examples:** Spreading rumours, posting embarrassing photos, sending threats.

3. Safe Online Habits (How to Protect Yourself)

1. **Use Antivirus Software:** Install good antivirus software (like Windows Defender, Avast) and keep it updated.
2. **Use Strong Passwords:** (Review) Use a mix of letters (upper/lower), numbers, and symbols.
3. **Think Before You Click:** Be very suspicious of emails and links. If in doubt, don't click.
4. **Use a Firewall:** This is a "digital wall" that blocks unauthorized access to your network (Windows has one built-in).
5. **Back Up Your Data:** Keep a copy of your important files (e.g., on an external hard drive or in the cloud, Week 8) so that if you are hit by ransomware, you don't have to pay.
6. **Be Private:** Set your social media (Instagram, Facebook) accounts to "Private."
7. **Report and Block:** (Review) Immediately report and block any cyberbullies or scammers.

Evaluation:

1. What is cybersecurity?
2. What is "malware"? Give one example.
3. What is "phishing"?
4. What is "cyberbullying"?
5. List three safe online habits.

Assignment:

1. What is the difference between a virus and phishing?
2. What is "ransomware"?
3. List three signs of a phishing email.
4. What is the first thing you should do if you are being cyberbullied?
5. What does antivirus software do?



WEEK 10: DATA PROTECTION

Topic: Data Protection

Breakdown (Subtopics):

- Meaning and importance
- Legal and ethical responsibilities



Performance Objectives: By the end of this lesson, students should be able to:

1. Define "Personal Data" and "Data Protection."

2. State three reasons why data protection is important.
3. Define "Consent."
4. Explain the legal responsibility of a company that collects data.
5. Explain the ethical responsibility of an individual regarding data.

Lesson Content:

1. Meaning of Data Protection

- **Personal Data:** This is any information that can be used to **identify you** as an individual.
 - *Examples:* Your full name, BVN (Bank Verification Number), NIN (National Identity Number), home address, phone number, email address, your photo, your medical records.
- **Data Protection:** This refers to the **laws, rules, and practices** that are designed to protect your personal data from being collected, used, or shared without your permission.

2. Importance of Data Protection Why should we care if someone has our data?

1. **To Prevent Identity Theft:** This is the *biggest* risk. A criminal who gets your NIN, BVN, and name can pretend to be you, open bank accounts, and take out loans in your name.
2. **To Protect Personal Privacy:** You have a right to privacy. You have the right to control who knows where you live or what your phone number is.
3. **To Prevent Scams and Fraud:** If scammers get your phone number or email (from a company's data breach), they will target you with phishing (Week 9) and scam messages.
4. **To Control Your Reputation:** To prevent people from using your data (like photos) to create fake profiles or spread rumours (cyberbullying).

3. Legal and Ethical Responsibilities

- **A. Legal Responsibilities (For Companies):**
 - In Nigeria, the main law is the **NDPR (Nigeria Data Protection Regulation)**. This law *forces* organizations (like banks, schools, hospitals, MTN, Facebook) that collect your data to follow strict rules:

2. **Consent:** They must get your *clear permission* (consent) before collecting your data. (This is the "I agree" box you tick).
3. **Security:** They must use strong security (like encryption) to protect your data from hackers.
4. **Purpose:** They can only use your data for the *specific reason* they told you (e.g., a bank can't sell your data to a marketing company).
5. **Breach Notification:** If they get hacked, they must *notify* you and the government.

- **B. Ethical Responsibilities (For Individuals):**

- **Ethics** = What is morally right or wrong.
- You also have an ethical duty to protect *other people's* data.
- *Example:* It is **unethical** (morally wrong) to:
 - Share your friend's phone number without their permission.
 - Post an embarrassing photo or video of a classmate.
 - Read your sibling's private messages or diary.
 - Use your parent's debit card online without asking.

Evaluation:

1. What is "personal data"? Give two examples.
2. What is "data protection"?
3. What is "identity theft"?
4. What does "consent" mean in data protection?
5. Is it right (ethical) to share your friend's phone number with someone else? Why or why not?

Assignment:

1. Define Personal Data and Data Protection.
2. List three reasons why data protection is important.
3. What is the full meaning of NDPR in Nigeria?
4. What are two *legal* responsibilities a company (like a bank) has when it collects your data?

5. What are two *ethical* responsibilities *you* have when handling your friends' or family's data?



WEEK 11: REVISION

Topic: Revision

Breakdown (Subtopics):

- Review of term's topics

Performance Objectives: By the end of this lesson, students should be able to:

1. Define the key concepts of the term (Network, Topology, Internet, Cybersecurity).
2. Explain the relationship between all the topics studied.
3. Discuss the importance of online safety in a networked world.

Lesson Content:

1. Summary of Term 1 Topics (The "Network & Security" Term):

- **Week 1 (Review):** We need Hardware and Software to use a computer. Online Safety is essential.
- **Week 2 (Networks):** A Network is interconnecting computers to share resources (like printers, files). Types: LAN (school), MAN (city), WAN (world/internet).
- **Week 3 (Components):** Networks are built with Servers (give resources), Clients (use resources), Switches (connect computers intelligently), Routers (connect networks), and Media (cables or Wi-Fi).
- **Week 4 (Topologies):** This is the network *layout*. Bus (one main cable), Star (central switch, most common), Ring (a circle).
- **Week 6 (Internet):** The Internet is the biggest WAN. The Web (WWW) is the collection of sites *on* the internet. We use Browsers (Chrome) to *view* sites and Search Engines (<https://www.google.com/search?q=Google.com>) to *find* sites. Email is a communication service.
- **Week 8 (Cloud):** The "Cloud" (e.g., Google Drive) uses the internet to store your files on remote servers, providing backup and accessibility.

- **Week 9 (Cybersecurity):** Because we are all connected, we need Cybersecurity to protect us from Malware (viruses), Phishing (fake emails), and Cyberbullying.
- **Week 10 (Data Protection):** Our Personal Data (NIN, BVN, name) is valuable and must be protected by laws (NDPR) and ethics to prevent identity theft.

2. Class Activity: Connecting the Dots (The Big Picture)

- We connect computers in a LAN (Week 2) using Cables and a Switch (Week 3) in a Star topology (Week 4).
- A Router (Week 3) connects our LAN to the Internet (Week 6).
- On the Internet, we use a Browser (Week 6) to access services like Cloud Storage (Week 8) and Email (Week 6).
- Because we are on the Internet, we are vulnerable to Phishing and Viruses (Week 9).
- Therefore, we must use Cybersecurity practices (Week 9) and understand Data Protection laws (Week 10) to stay safe online (Week 1).

Evaluation: This week's evaluation is the First Term Examination.

Assignment:

1. Study all notes from Week 1 to Week 10 for the examination.
2. Draw a diagram showing the difference between a LAN and a WAN connected by a router.
3. Draw a Star topology.
4. What is the difference between a Browser and a Search Engine?
5. What are the three biggest online dangers you learned about this term? (e.g., Phishing, Cyberbullying, Identity Theft).

SECOND TERM – JSS 2 DIGITAL TECHNOLOGIES LESSON NOTES

WEEK 1: INTRODUCTION TO PROGRAMMING

Topic: Introduction to Programming

Breakdown (Subtopics):

- Meaning and importance
- Types of programming languages
- Examples: Scratch, Python



Performance Objectives: By the end of this lesson, students should be able to:

1. Define "Programming" and "Program."
2. State at least three reasons why programming is important.
3. Differentiate between Low-Level and High-Level programming languages.
4. Differentiate between text-based and block-based (visual) programming.
5. Give an example of a block-based language (Scratch) and a text-based language (Python).

Lesson Content:

1. Meaning and Importance

- **What is a Program?**

- A program is a set of **step-by-step instructions**, written in a special language, that tells a computer *exactly* what to do to perform a task.

- **What is Programming (or Coding)?**

- Programming is the **process of writing, testing, and maintaining** these instructions (the program). The person who writes a program is called a **Programmer** or a **Coder**.

- **Analogy:** A computer is like a very obedient but "dumb" robot. It only does *exactly* what you tell it. A "program" is like a perfect recipe, and "programming" is the act of writing that recipe so the robot (computer) can cook a meal.

- **Importance of Programming:**

1. **It Powers All Technology:** Every software you use (Windows, Google Chrome, WhatsApp, video games) is a program. Without programming, computers are just useless boxes of metal and plastic.

2. **Problem Solving:** Programming teaches you how to think logically and break down large, complex problems (like "build a game") into small, simple, solvable steps.

3. **Automation:** It allows us to automate repetitive and boring tasks (e.g., calculating all student grades, sending 1,000 emails).

4. **Career Opportunities:** It is one of the most in-demand skills in the world (e.g., Software Engineer, Web Developer).

2. Types of Programming Languages A programming language is the special vocabulary and set of rules used to write instructions for a computer.

- **A. Low-Level Languages:**

- **Definition:** Languages that are very *close* to the computer's own hardware language (binary: 0s and 1s).
- **Characteristics:** Very difficult for humans to read and write, but very fast for the computer.
- **Examples:**
 - **Machine Language (First Gen):** Made of 0s and 1s (e.g., 01011001 11000011).
 - **Assembly Language (Second Gen):** Uses short codes (mnemonics) like ADD, SUB, MOV.

- **B. High-Level Languages:**

- **Definition:** Languages that are *closer* to human language (like English). They are easier for humans to read, write, and understand.
- **Characteristics:** They must be *translated* (by a compiler or interpreter) into low-level machine language before the computer can run them.
- **Examples:** Python, Java, C++, BASIC.

3. Examples: Block-Based vs. Text-Based (Both are High-Level)

- **A. Block-Based (Visual) Programming:**

- **Definition:** A type of programming where the user connects (drags and drops) graphical blocks, like "digital Lego bricks," to build a program.
- **Target:** Excellent for beginners, as it removes the frustration of typing errors (syntax).
- **Example: Scratch.**
 - To make a character move, you drag a "Move 10 steps" block.

- **B. Text-Based Programming:**

- **Definition:** The traditional type of programming where the user must type all the commands and instructions using precise keywords and punctuation (this is called **syntax**).
- **Example: Python.**
 - To show a message, you must type: `print("Hello, World!")`
 - If you misspell print or forget the brackets, the program will crash (an "error").

Evaluation:

1. What is a "program"?
2. What is "programming"?
3. State two reasons why programming is important.
4. What is the main difference between a high-level and a low-level language?
5. What is the main difference between block-based programming (like Scratch) and text-based programming (like Python)?

Assignment:

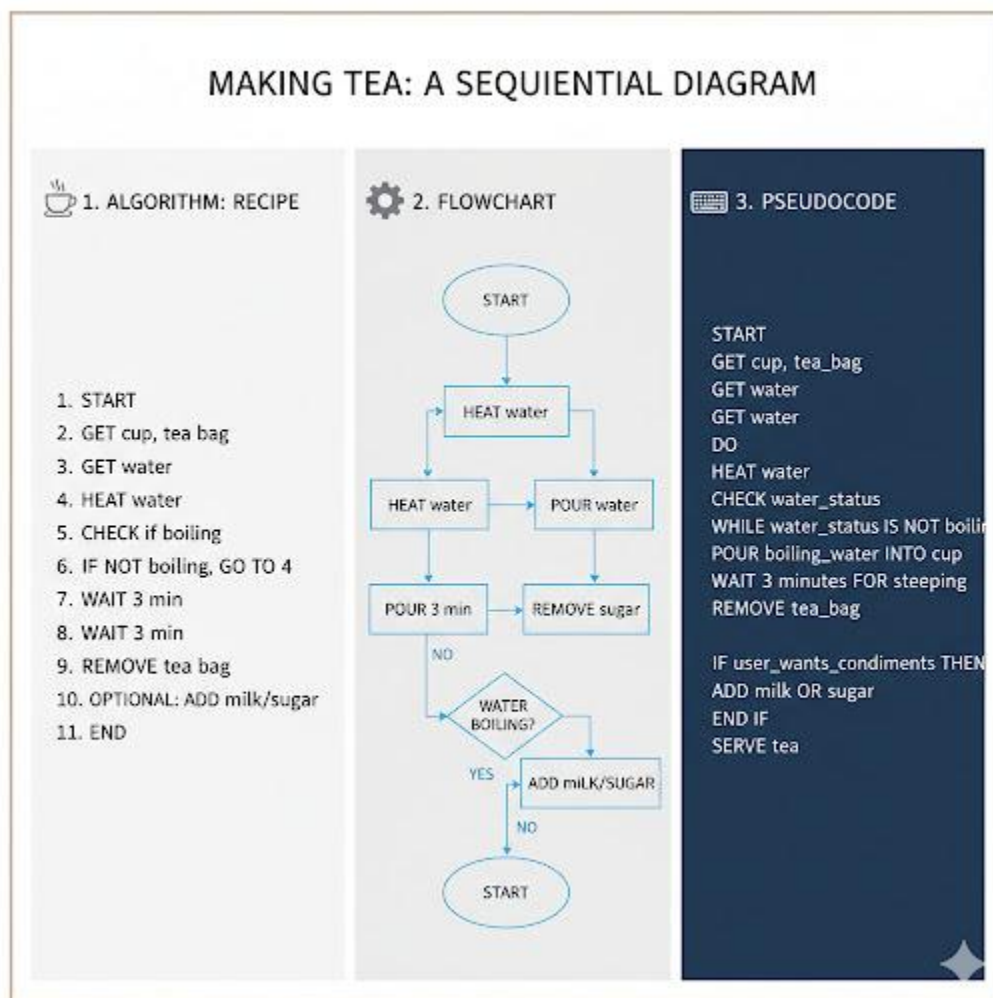
1. Define Programming and Programmer.
2. List three reasons why programming is a valuable skill to learn.
3. In a table, list two differences between High-Level and Low-Level languages.
4. What is "syntax" in programming? Which type of programming (block or text) depends heavily on correct syntax?
5. What is the name of the block-based language we will be using?

WEEK 2: BASIC ALGORITHMS

Topic: Basic Algorithms

Breakdown (Subtopics):

- Meaning of algorithm
- Steps in problem solving
- Flowcharts and pseudocode



Performance Objectives: By the end of this lesson, students should be able to:

1. Define "Algorithm" and give a real-life example.
2. List the basic steps in solving a problem with a computer.

3. Define "Flowchart" and identify four basic flowchart symbols.
4. Define "Pseudocode."
5. Write a simple algorithm using pseudocode or a flowchart.

Lesson Content:

1. Meaning of Algorithm

- **Definition:** An **algorithm** is a **finite, step-by-step set of instructions** designed to perform a specific task or solve a particular problem.
- **Analogy:** It is the "recipe" for solving a problem. The program (Week 1) is the algorithm written in a language the computer understands.
- **Real-Life Examples:**
 - **Making Tea:** 1. Get a cup. 2. Put a teabag in. 3. Boil water. 4. Pour water into cup. 5. Add milk/sugar. 6. Stir.
 - **Brushing Teeth:** 1. Get brush. 2. Apply toothpaste. 3. Brush teeth (all surfaces). 4. Rinse mouth.
- **Characteristics:** An algorithm must be **clear** (unambiguous), **finite** (it must end), and **correct** (it must solve the problem).

2. Steps in Problem Solving (Programming) You don't just start coding. You must plan.

1. **Define/Understand the Problem:** What *exactly* do you need to do? (e.g., "I need to find the average of three numbers").
2. **Plan the Solution (Algorithm):** Design the step-by-step instructions. This is where you use flowcharts or pseudocode.
3. **Code (Write the Program):** Translate your algorithm (plan) into a programming language (like Scratch or Python).
4. **Test and Debug:** Run the program. Does it work? Does it give the right answer? If not, find the "bugs" (errors) and fix them (this is called *debugging*).
5. **Document:** Write comments and explanations so others (or you, later) can understand your code.

3. Planning Tools: Flowcharts

- **Definition:** A **flowchart** is a **graphical or diagrammatic representation** of an algorithm. It uses standard symbols connected by arrows to show the *flow* of logic.
- **Basic Symbols:**
 - **Oval (Terminator):** START / END. Every flowchart begins and ends with this.
 - **Parallelogram (Input/Output):** Get Number, Print Answer. Used for getting data or showing results.
 - **Rectangle (Process):** Sum = A + B. Used for any calculation or action.
 - **Diamond (Decision):** Is A > B?. Used to ask a "Yes/No" question (for conditions, Week 6).
 - **Arrows (Flow Lines):** Show the direction of the steps.

4. Planning Tools: Pseudocode

- **Definition:** **Pseudocode** means "fake code." It is a way of writing your algorithm using a simple, English-like structure *before* you write it in a real programming language. It has no strict syntax rules.
- **Example:** Algorithm to find the sum of two numbers.

Flowchart Version	Pseudocode Version
START	START
↓	Get first number (A)
Get A	Get second number (B)
↓	Calculate: Sum = A + B
Get B	Display Sum
↓	END
Sum = A + B	
↓	
Display Sum	
↓	
END	

Evaluation:

1. What is an algorithm?
2. Give one real-life example of an algorithm.
3. What is a flowchart?
4. What is pseudocode?
5. What shape is used for "START/END" in a flowchart? What shape is for a "Process/Calculation"?

Assignment:

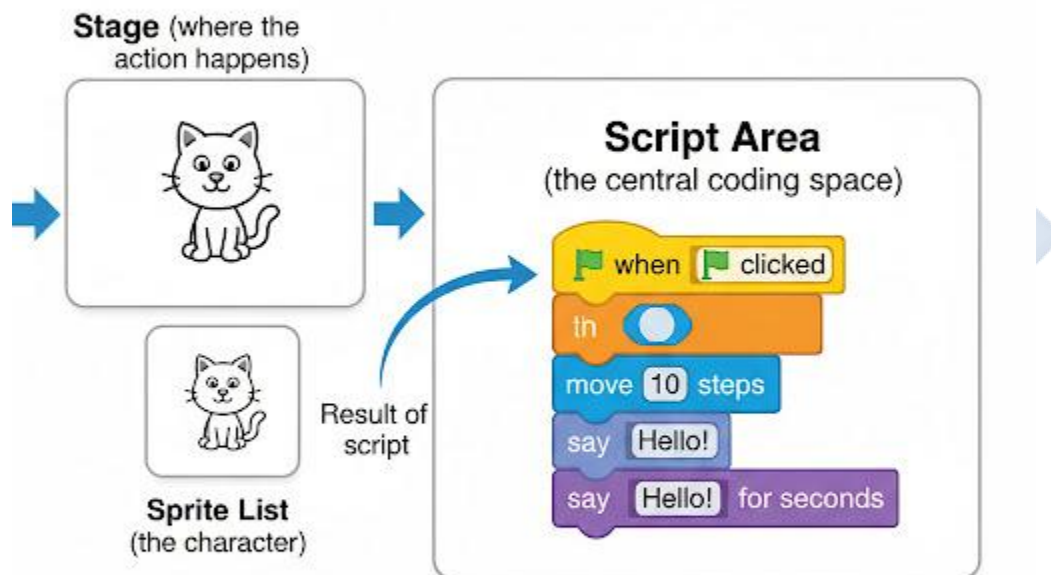
1. Define Algorithm, Flowchart, and Pseudocode.
2. List the main steps in problem solving.
3. What is the "diamond" symbol used for in a flowchart?
4. Write **pseudocode** for an algorithm that takes a person's Year of Birth and the Current Year and then calculates and displays their Age.
5. Draw a **flowchart** for the same algorithm in question 4.

WEEK 3: BLOCK CODING

Topic: Block Coding

Breakdown (Subtopics):

- Understanding block-based coding (Scratch)
- Creating basic animations



Performance Objectives: By the end of this lesson, students should be able to:

1. Identify the main parts of the Scratch interface.
2. Define "Sprite" and "Stage."
3. Define "Script" as a set of connected blocks.

4. Create a simple script to make a Sprite move and talk.
5. Explain how to create a basic animation.

Lesson Content:

1. What is Block-Based Coding? (Review) Block-based (or visual) coding uses graphical "blocks" that snap together. This allows you to focus on the *logic* (the algorithm) without worrying about *syntax* (typing errors). Our main tool is **Scratch**.

2. The Scratch Interface When you open Scratch, you will see four main areas:

1. The Stage (Top-Left):

- This is the "screen" where your program, game, or animation happens.
- The green flag (Go) and red stop sign (Stop) are here.

2. The Sprite List (Bottom-Left):

- **Sprite:** A *Sprite* is a character or object in your program (e.g., the default orange cat).
- This area lists all the Sprites in your project.

3. The Block Palette (Middle):

- This is the "toolbox." It contains all the code blocks, organized by colour and function:
 - Motion (blue): Move, turn, go to...
 - Looks (purple): Say, think, change costume...
 - Sound (magenta): Play sound...
 - Events (yellow): When green flag clicked, When space key pressed...
 - Control (orange): Wait, repeat, if...then...

4. The Script Area (Right):

- This is your "workspace." You **drag blocks** from the Palette and **snap them together** here to build your program (your "script").

3. What is a Script? A **script** is a stack of blocks that are connected together. Each Sprite has its own Script Area and can have its own scripts.

4. Creating a Basic Animation (Your First Script)

- **Goal:** Make the cat (Sprite1) move, say "Hello!", and turn.
- **Algorithm (Pseudocode):**
 1. START (when the green flag is clicked)
 2. Move 100 steps
 3. Say "Hello!" for 2 seconds
 4. Turn 90 degrees
 5. END (script finishes)
- **Scratch Script (How to build it):**
 1. Go to the Events (yellow) palette and drag the When green flag clicked block into the Script Area. This is the *starter* for almost every script.
 2. Go to the Motion (blue) palette. Drag a move 10 steps block and snap it *under* the "when flag clicked" block. Change "10" to "100".
 3. Go to the Looks (purple) palette. Drag a say "Hello!" for 2 seconds block and snap it under the "move" block.
 4. Go back to the Motion (blue) palette. Drag a turn (clockwise) 15 degrees block and snap it at the bottom. Change "15" to "90".
- **Run:** Click the green flag on the Stage. The cat will perform the actions in order. This is a simple animation!

Evaluation:

1. What is a "Sprite" in Scratch?
2. What is the "Stage" in Scratch?
3. What is the "Block Palette"?
4. What is a "Script"?
5. Which *Event* block is most commonly used to start a script? (When green flag clicked).

Assignment:

1. Draw a simple diagram of the Scratch interface and label the 4 main areas.
2. What is the function of the Motion (blue) blocks?

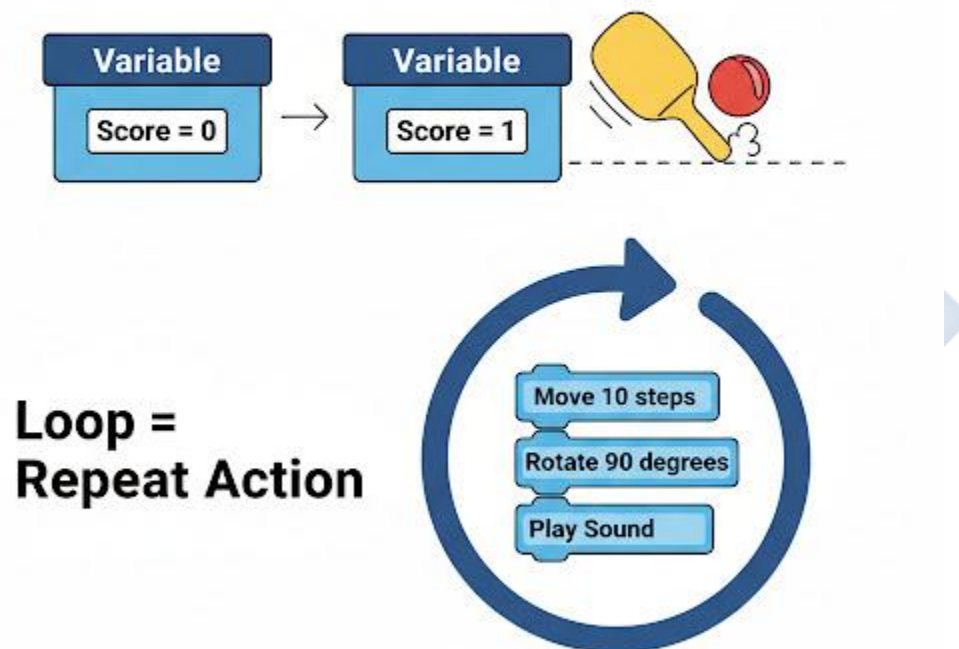
3. What is the function of the Looks (purple) blocks?
4. Write the *pseudocode* for an algorithm where a Sprite moves, waits for 1 second, and then says "I'm thinking..."
5. Describe (or draw) the Scratch *script* (the blocks) you would build to make the algorithm in Question 4 work.



WEEK 4: VARIABLES IN PROGRAMMING

Topic: Variables in Programming **Breakdown (Subtopics):**

- Meaning and uses of variables
- Creating and naming variables in Scratch



Performance Objectives: By the end of this lesson, students should be able to:

1. Define "Variable" using an analogy.
2. State two reasons why variables are used in programming.
3. Create a variable in Scratch.

4. Use Scratch blocks to set and change the value of a variable.
5. Create a simple script that uses a variable (e.g., a scorekeeper).

Lesson Content:

1. Meaning of Variable

- **Definition:** A **variable** is a placeholder in a computer's memory used to **store a piece of information**. The information it holds (its "value") *can change* or *vary* while the program is running.
- **Analogy:** A variable is like a **labelled box** or a "container."
 - The **Name** of the variable is the *label* on the box (e.g., "Score").
 - The **Value** of the variable is the *item inside* the box (e.g., the number 10).

2. Uses of Variables

Variables are essential in programming. We use them to store:

- A player's **score** in a game.
- A player's **name**.
- The number of **lives** a player has left.
- The **speed** of a character.
- The answer to a math question.

3. Creating and Naming Variables in Scratch

- **How to Create:**
 1. Go to the Variables (dark orange) palette.
 2. Click the "Make a Variable" button.
 3. A box will appear. Type a name for your variable (e.g., score).
 4. Choose "For all sprites" (for now).
 5. Click OK.
- **What happens:**
 - Your new variable (score) will appear in the palette with its own special blocks:
 - score (the oval-shaped reporter block)

- set [score] to (0) (sets the value)
- change [score] by (1) (adds to the value)
- A "monitor" for your variable will also appear on the Stage, showing its name and current value.

4. Using Variables (Example: A Scorekeeper)

- **Goal:** Create a program where the score starts at 0 and goes up by 1 every time you click the Sprite.
- **Algorithm:**
 1. START (when green flag is clicked)
 2. Set the 'score' variable to 0
 3. START (when this sprite is clicked)
 4. Add 1 to the 'score' variable
- **Scratch Script (Two separate scripts in the same Script Area):**
 - **Script 1 (Initialization):**
 1. Drag When green flag clicked (from Events).
 2. Drag set [score] to (0) (from Variables).
 - *This script resets the score to 0 every time the game starts.*
 - **Script 2 (The Action):**
 0. Drag When this sprite clicked (from Events).
 1. Drag change [score] by (1) (from Variables).
 - *This script adds 1 to the score every single time the user clicks the cat.*

Evaluation:

1. What is a variable? Use the "box" analogy.
2. What is the "value" of a variable?
3. List two things we use variables for in a game.
4. In Scratch, what *colour* is the Variables palette?

5. What is the difference between the set [score] to (0) block and the change [score] by (1) block?

Assignment:

1. Define "Variable" in your own words.
2. What are the steps to "Make a Variable" in Scratch?
3. Write the *pseudocode* for a simple "clicker" game (like the one in the lesson).
4. Draw the two Scratch *scripts* (the blocks) needed to make the clicker game work.
5. Imagine you are making a game with "Lives." What would you name your variable?
What value would you set it to at the start of the game? (e.g., 3 or 5).

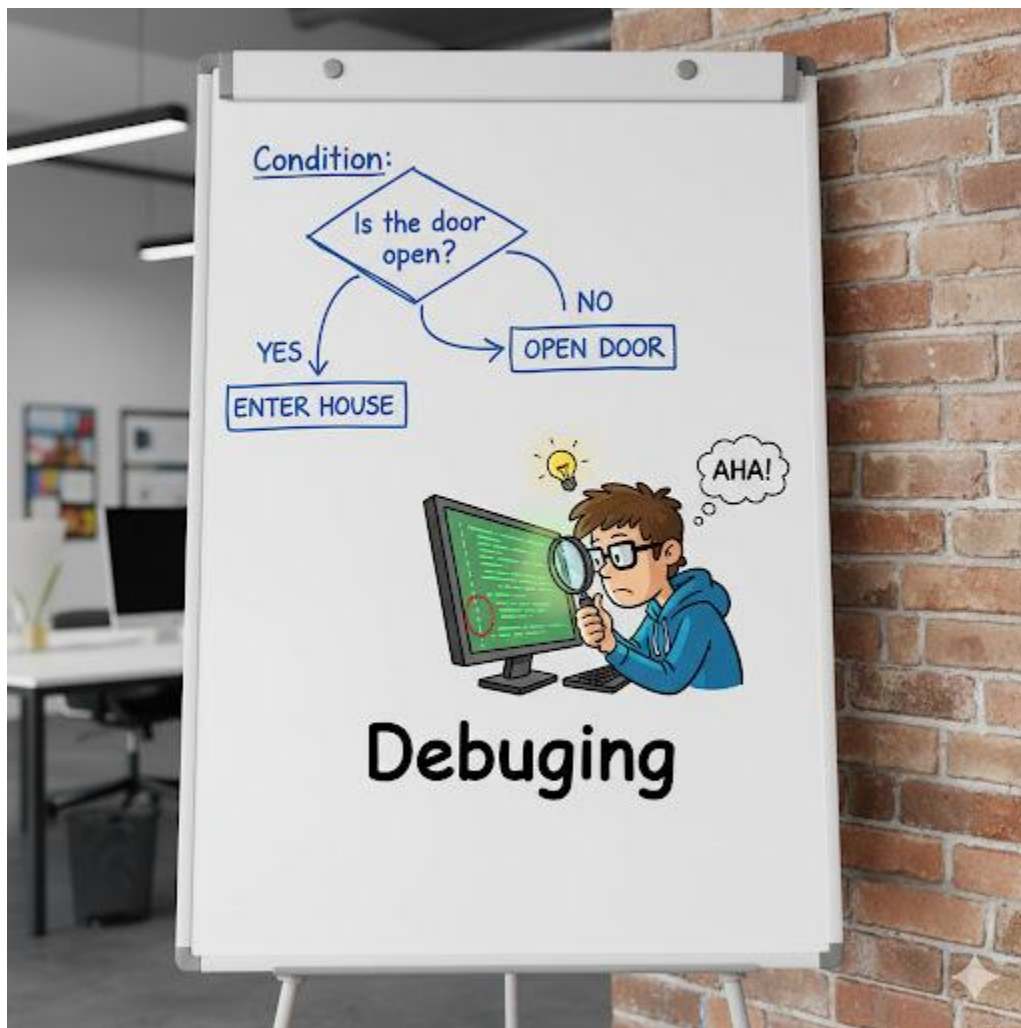


WEEK 6: CONTROL WITH CONDITIONS

Topic: Control with Conditions

Breakdown (Subtopics):

- Conditional statements (IF, ELSE)
- Decision-making in programs



Performance Objectives: By the end of this lesson, students should be able to:

1. Define "Conditional Statement."
2. Identify the "IF...THEN" and "IF...THEN...ELSE" structures.
3. Identify the hexagonal "sensing" blocks in Scratch.

4. Use the if...then block in a script.
5. Use the if...then...else block in a script.

Lesson Content:

1. What is a Conditional Statement?

- **Definition:** A conditional statement (or "condition") is a feature of programming that allows your program to make **decisions**. It checks if a certain condition is **True** or **False** and then performs different actions based on the result.
- **Analogy:** It's like your brain works:
 - "IF it is raining, THEN I will take an umbrella." (Simple condition)
 - "IF it is raining, THEN I will take an umbrella, ELSE I will wear my sunglasses." (A two-part condition)

2. Conditions in Scratch (Sensing)

- To make a decision, you first need to "sense" something.
- In Scratch, the Sensing (light blue) palette contains hexagonal-shaped blocks that ask "True/False" questions.
- **Examples:**
 - <touching [mouse-pointer]?>
 - <key [space] pressed?>
 - <color [#] is touching [#]?>
 - <() > (50)> (from Operators - green)

3. The "IF...THEN" Block

- **Scratch Block:** if < > then
- **Function:** This is a one-part decision. It checks the condition in the hexagonal slot.
 - If the condition is TRUE, it will run the blocks *inside* the "C" shape.
 - If the condition is FALSE, it *skips* those blocks and continues.
- **Example Script:** Make the cat hide when it touches the mouse pointer.

1. When green flag clicked (Events)
 2. forever (Control) - *We need a loop to keep checking!*
 3. { (Drag the if...then block inside the forever loop)
 4. Drag <touching [mouse-pointer]?> (Sensing) into the if block's slot.
 5. Drag hide (Looks) *inside* the "C" shape. }
- **Result:** The cat will run the loop forever. The *moment* the mouse touches it (condition becomes TRUE), it will run the hide block and disappear.

4. The "IF...THEN...ELSE" Block

- **Scratch Block:** if <> then ... else ...
 - **Function:** This is a two-part decision. It *always* does one of two things.
 - If the condition is TRUE, it runs the blocks in the *first* "C" shape.
 - If the condition is FALSE, it skips the first part and runs the blocks in the *second* "C" shape (the "else" part).
 - **Example Script:** Make the cat say "Ouch!" when touching the wall, *or* "Safe!" when not.
1. When green flag clicked
 2. forever
 3. { (Drag the if...then...else block inside)
 4. Drag <touching [edge]?> (Sensing) into the if block's slot.
 5. Drag say "Ouch!" for 0.5 secs (Looks) into the *first* "C" shape.
 6. Drag say "I am safe!" for 0.5 secs (Looks) into the *second* ("else") "C" shape. }
- **Result:** The cat will constantly check: Is it touching the edge? **If YES**, it says "Ouch!". **If NO**, it says "I am safe!"

Evaluation:

1. What is a conditional statement? What does it allow a program to do?
2. What is the difference between an if...then block and an if...then...else block?
3. What *shape* are the condition blocks in Scratch (e.g., from the Sensing palette)?
4. In the if...then...else block, when does the "else" part run?
5. What Sensing block would you use to check if the space bar is being pressed?

Assignment:

1. Define "Conditional Statement" and give a real-life (non-computer) example.
2. What is the function of the Sensing (light blue) palette?
3. Write the *pseudocode* for an algorithm that checks if a score is greater than 10. If it is, display "You Win!".
4. Draw the Scratch *script* (the blocks) for your algorithm in Question 3.
5. Now, write the *pseudocode* for an algorithm that checks if a score is greater than 10. If it is, display "You Win!", *else* display "Try Again!".

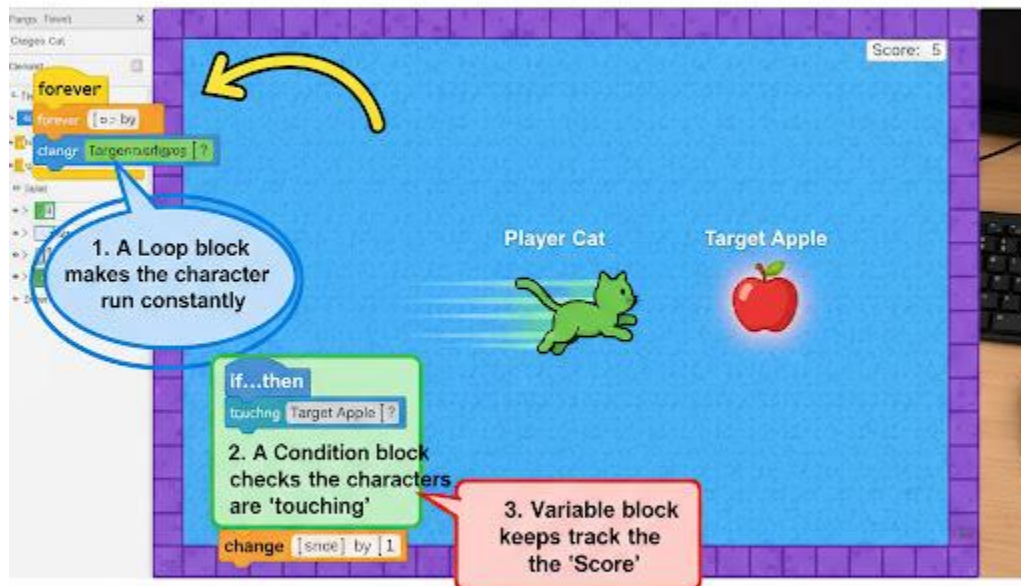


WEEK 8: LOOPS IN PROGRAMMING

Topic: Loops in Programming Breakdown (Subtopics):

- Meaning of loops
- Types: for loop, while loop
- Applications in coding

Game Logic: Combining Variables, Loops and Conditions



Performance Objectives: By the end of this lesson, students should be able to:

1. Define "Loop."
2. Explain why loops are a fundamental concept in programming.
3. Identify the "FOR" loop (definite loop) in Scratch.

4. Identify the "WHILE" loop (indefinite loop) in Scratch.
5. Create a script that uses a loop to repeat an action.

Lesson Content:

1. Meaning of Loops (Iteration)

- **Definition:** A **loop** (or *iteration*) is a control structure in programming that allows you to **repeat** a set of instructions multiple times.
- **Importance:** Loops are fundamental. They save you time and make your code efficient.
- **Analogy (Without a loop):** To make a Sprite move in a square, you would need 8 blocks:
 - move 100 steps
 - turn 90 degrees
 - move 100 steps
 - turn 90 degrees
 - move 100 steps
 - turn 90 degrees
 - move 100 steps
 - turn 90 degrees
- **Analogy (With a loop):**
 - Repeat 4 times:
 - move 100 steps
 - turn 90 degrees
 - This is only 3 blocks! It's cleaner, faster, and easier to change (e.g., to a triangle, just Repeat 3 times and turn 120 degrees).

2. Types of Loops

- **A. "FOR" Loop (Definite Loop)**
 - **Definition:** A loop that repeats a block of code a **fixed, known number of times**. You *define* how many times it should run.
 - **Scratch Block:** repeat (10) (from the Control palette).
 - **Use:** When you know exactly how many repetitions you need.

- **Example Script (The Square):**
 1. When green flag clicked
 2. repeat (4) (from Control)
 3. { (Drag the next two blocks *inside* the repeat loop)
 4. move (100) steps (from Motion)
 5. turn (90) degrees (from Motion) }
 6. wait (1) sec (from Control, *after* the loop)
- **B. "WHILE" Loop (Indefinite Loop)**
 - **Definition:** A loop that repeats *as long as* (or *until*) a certain **condition** is true. You do *not* know how many times it will run.
 - **Scratch Blocks:**
 - forever (This is a "while TRUE" loop. It *never* stops).
 - repeat until < > (This is a "while NOT" loop. It repeats *until* the condition becomes TRUE, then it stops).
 - **Use:** When you want the code to keep running based on a condition (like a game loop).
 - **Example Script (Move until you hit the wall):**
 0. When green flag clicked
 1. go to x:(-200) y:(0) (Start at the left edge)
 2. repeat until <touching [edge]?> (from Control and Sensing)
 3. { (Drag this block *inside* the loop)
 4. move (10) steps (from Motion) }
 5. say "I'm at the edge!" (from Looks, *after* the loop)
 - **Result:** The cat will keep moving 10 steps, over and over, *until* it hits the edge, at which point the loop stops and it says "I'm at the edge!"

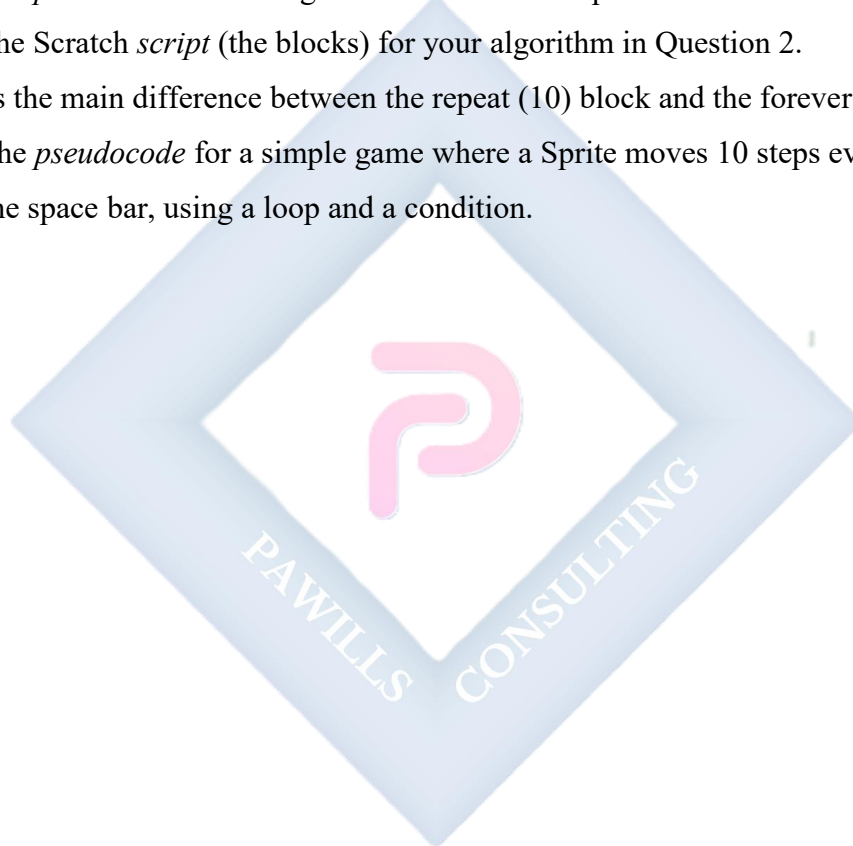
Evaluation:

1. What is a "loop" in programming?
2. Why are loops so important (what do they save)?

3. What is a "FOR" loop (or definite loop)?
4. What is a "WHILE" loop (or indefinite loop)?
5. Which Scratch block is a "FOR" loop (repeat (10)) and which is a "WHILE" loop (repeat until <>)?

Assignment:

1. Define "Loop" (or "Iteration").
2. Write the *pseudocode* for an algorithm that uses a loop to make a cat "meow" 5 times.
3. Draw the Scratch *script* (the blocks) for your algorithm in Question 2.
4. What is the main difference between the repeat (10) block and the forever block?
5. Write the *pseudocode* for a simple game where a Sprite moves 10 steps every time you press the space bar, using a loop and a condition.



WEEK 9: GAME DEVELOPMENT WITH VISUAL PROGRAMMING

Topic: Game Development with Visual Programming

Breakdown (Subtopics):

- Steps in game creation
- Designing a simple interactive game



Performance Objectives: By the end of this lesson, students should be able to:

1. List the basic steps in game creation.
2. Combine variables, conditions, and loops to create a simple game.
3. Add a "win" or "lose" condition to their game.

4. Create an interactive game where one sprite is controlled by the user.

Lesson Content:

1. Steps in Game Creation (Game Design Process)

1. **Brainstorm (Plan):** What is your game? What is the *goal*? (e.g., A "chase" game. Goal: catch the mouse. A "maze" game. Goal: get to the apple).
2. **Design Assets:** Who are the *Sprites* (characters)? What does the *Stage* (background) look like?
3. **Build Scripts (Code):** Add the blocks. How does the player move? How do you score? How do you win?
4. **Test and Debug:** Play the game. Does it work? Is it fun? What is broken? (This is Debugging, Week 10).
5. **Refine:** Make it better (add sounds, new levels, etc.).

2. Designing a Simple Interactive Game: "Cat and Mouse"

- **Goal:** The player controls the Cat. They must catch the Mouse to win.
- **Assets:**
 - **Sprite 1:** The "Cat" (or any sprite).
 - **Sprite 2:** The "Mouse" (or any sprite).
 - **Stage:** Any background (e.g., "Bedroom").

3. Building the Scripts (Combining our skills)

- **Script for the Cat (The Player):**
 - We want the player to control the cat with the mouse pointer.
 - **Algorithm:**
 1. START (when green flag clicked)
 2. Loop forever:
 3. Make the cat go to the position of the mouse-pointer.
 - **Scratch Script (for the *Cat* Sprite):**

1. When green flag clicked (Events)
 2. forever (Control)
 3. { go to [mouse-pointer] (Motion) }
- **Script for the Mouse (The Target):**
 - We want the mouse to check *if* it is being caught by the cat.
 - **Algorithm:**
 1. START (when green flag clicked)
 2. Make the mouse show up.
 3. Loop forever:
 4. Check: IF the mouse is touching the "Cat" sprite, THEN:
 5. Say "You caught me!" for 2 seconds.
 6. Hide the mouse sprite.
 7. Stop all scripts (end the game).
 - **Scratch Script (for the *Mouse* Sprite):**
 1. When green flag clicked
 2. show (Looks)
 3. forever (Control)
 4. { if <touching [Cat]?> (Sensing, select the cat sprite's name) then
 5. { say "You caught me!" for 2 secs (Looks)
 6. hide (Looks)
 7. stop [all] (Control) } }
 - **Putting it all together:** When you click the green flag, you can now "fly" the cat around with your mouse. The moment you fly over the mouse sprite, it will say "You caught me!", hide, and the game will end. **You have made an interactive game!**

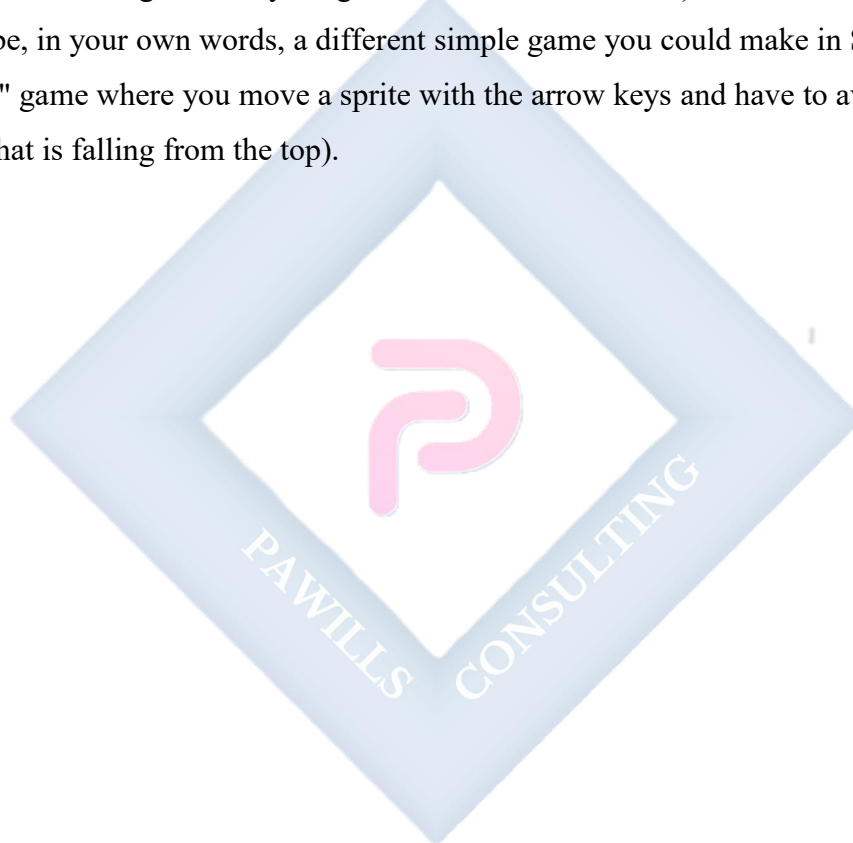
Evaluation:

1. List the first three steps in game creation.
2. In our "Cat and Mouse" game, how did we make the Cat interactive (controlled by the player)?
3. In our game, what *Sensing* block did we use?
4. What *Loop* block did we use?

5. What was the "win condition" in our game? (Touching the mouse).

Assignment:

1. List the 5 steps of the game design process.
2. Write the *pseudocode* for the **Cat's** script in our game.
3. Write the *pseudocode* for the **Mouse's** script.
4. **Challenge:** How would you add a score (a Variable) to this game? (e.g., set score to 0 at the start, and change score by 1 right before the mouse hides).
5. Describe, in your own words, a different simple game you could make in Scratch (e.g., a "dodge" game where you move a sprite with the arrow keys and have to avoid another sprite that is falling from the top).

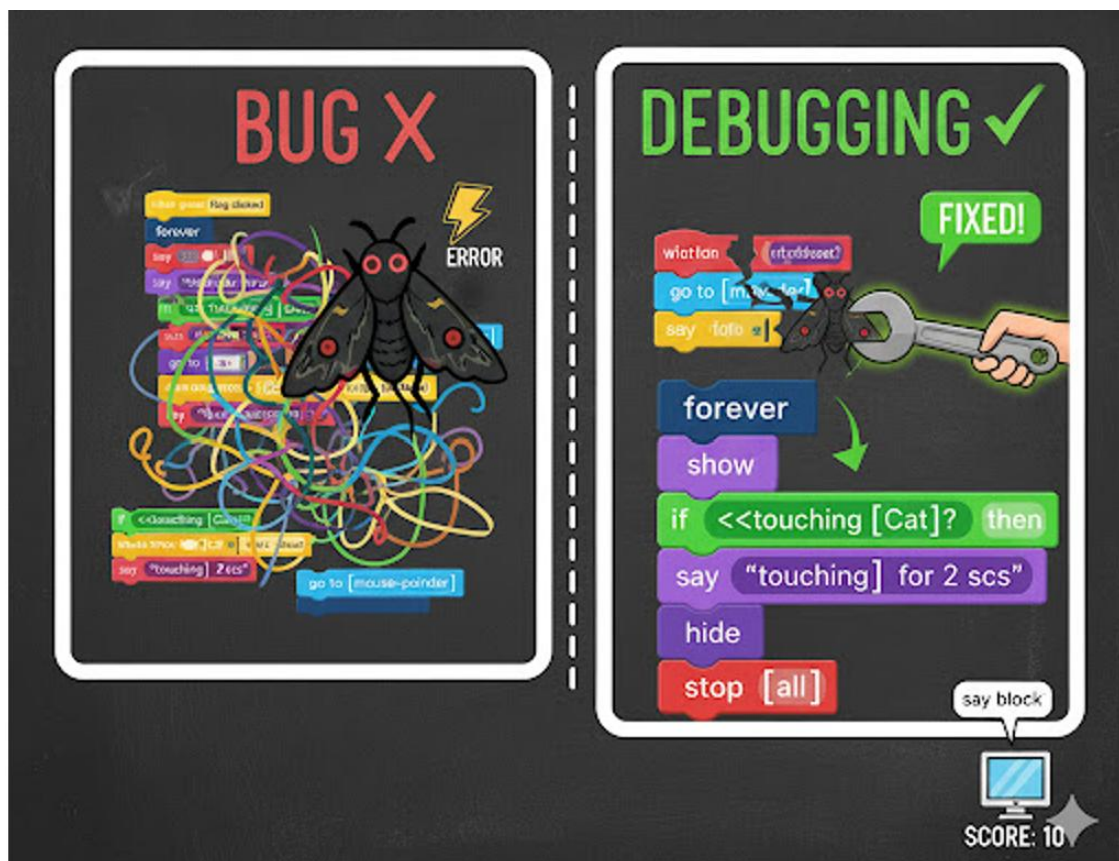


WEEK 10: DEBUGGING

Topic: Debugging

Breakdown (Subtopics):

- Meaning and importance
- Finding and fixing errors in code



Performance Objectives: By the end of this lesson, students should be able to:

1. Define "Bug" and "Debugging."
2. State why debugging is a normal and essential part of programming.
3. Identify a simple bug in a Scratch script.
4. Describe at least two methods for finding and fixing bugs.

Lesson Content:

1. Meaning of "Bug" and "Debugging"

- **Bug:** A **bug** is an **error, mistake, or fault** in a computer program that causes it to behave in unexpected ways, produce an incorrect result, or crash.
- **Debugging:** **Debugging** is the process of **finding and fixing** these bugs (errors) in your code.
- **Origin:** The term was (famously, perhaps apocryphally) coined in the 1940s when a real **moth** (a bug) got stuck in a computer's hardware (a relay) and caused it to stop working.

2. Importance of Debugging

- **Fact:** ALL programmers, from beginners to experts, make mistakes. Writing code that works perfectly the first time is extremely rare.
- **Importance:** Debugging is not a "punishment" for bad coding; it is a *normal* and *essential* part of the programming process (Step 4: Test and Debug).
- Being a good programmer is not about *never* making bugs, it's about being *good* at *finding* and *fixing* them.

3. Types of Bugs (Errors)

1. **Syntax Errors:** (Not in Scratch). This is a *typing* or *grammar* error, like misspelling a command (prnt instead of print) or forgetting a bracket. The program won't even run.
2. **Logic Errors:** (Very common in Scratch). The program *runs*, but it doesn't do what you *wanted* it to do. The logic of your algorithm is wrong.

4. Finding and Fixing Bugs (Debugging Techniques)

- **Example Bug:**
 - You make the "Cat and Mouse" game (Week 9). You click the green flag, and the score *immediately* goes to 1 and the mouse says "You caught me!" before you even move.

- **The Bug:** You forgot to move the Cat and Mouse apart at the start. They are touching each other *at the beginning*, so the if <touching [Cat]?> condition is TRUE on the very first frame.
 - **The Fix:** Add a go to x:(100) y:(100) block to the Mouse's "when flag clicked" script, and a go to x:(-100) y:(-100) block to the Cat's script, to make sure they start on opposite sides of the stage.
- **Debugging Techniques:**

1. **Test One Block at a Time:** If a long script isn't working, pull it apart. Click on each block individually to see what it *actually* does.
2. **Use "Say" Blocks:** This is the best way to debug variables. If you don't know what value your score variable has, add a say [score] block (from Looks and Variables) into your loop. The Sprite will "say" its score, and you can see if it's counting correctly.
3. **Read Your Code Slowly:** Read your code, block by block, and pretend *you* are the computer. What did you *tell* it to do? What did it *actually* do?
4. **Ask a Friend:** Sometimes a fresh pair of eyes can see a mistake you missed.

Evaluation:

1. What is a "bug" in programming?
2. What is "debugging"?
3. Why is debugging a normal part of programming?
4. What is a "logic error"?
5. What is one technique you can use to find a bug in a Scratch script? (e.g., using "say" blocks).

Assignment:

1. Define "Bug" and "Debugging."
2. What is the difference between a syntax error and a logic error?
3. Why is a "logic error" often harder to find than a "syntax error"?

4. You have a script: When green flag clicked, set [score] to (0). It *doesn't* reset the score. What is the *likely* bug? (Answer: The user is probably not clicking the green flag, or another script is changing the score).
5. Describe a bug you have had in one of your Scratch projects (or one you can imagine) and explain how you would *fix* it.



WEEK 11: REVISION

Topic: Revision

Breakdown (Subtopics):

- Review of programming concepts

Performance Objectives: By the end of this lesson, students should be able to:

1. Define the five key concepts of programming (Algorithm, Variable, Condition, Loop, Debugging).
2. Explain how these five concepts work together to create a program.
3. Review all Scratch block categories and their functions.

Lesson Content:

1. Summary of Term 2 Topics (The "Programming" Term): This term, we learned the five "big ideas" of all programming:

- **Week 1 (Programming):** A program is a set of instructions. We can use block-based (Scratch) or text-based (Python).
- **Week 2 (Algorithms):** An algorithm is the *plan* or *recipe* for a program. We plan using flowcharts (diagrams) and pseudocode (fake code).
- **Week 4 (Variables):** A variable is a "box" for storing data that can change.
 - *Scratch Blocks:* set [var] to (0), change [var] by (1).
- **Week 6 (Conditions):** Conditions are "decision-making" structures. They check if something is TRUE or FALSE.
 - *Scratch Blocks:* if < > then, if < > then, else.
- **Week 8 (Loops):** Loops are for *repeating* code.
 - *Scratch Blocks:* repeat (10) (a "for" loop), forever / repeat until < > (a "while" loop).
- **Week 10 (Debugging):** A bug is an error. Debugging is the normal process of finding and fixing errors.

2. Class Activity: Connecting the Dots (How to make a game) Let's review the "Cat and Mouse" game from Week 9 and see how it used *all* our concepts:

1. **Algorithm (Plan):** "We need a player (Cat) to follow the mouse. We need a target (Mouse) to check *if* it is touching the Cat. *If* it is, the game ends."
2. We wrote **Scripts** (the program) in Scratch.
3. We used a variable for score (optional, from Assignment).
4. We used an *indefinite* loop (forever) to *repeat* our checks.
5. Inside the loop, we used a condition (if <touching [Cat]?>) to make a *decision*.
6. When the game didn't work the first time (the "start-touching" bug), we used Debugging (by adding "go to" blocks) to fix it.

All five concepts work together to make a single, simple program.

Evaluation: This week's evaluation is the Second Term Examination.

Assignment:

1. Study all notes from Week 1 to Week 10 for the examination.
2. Define Algorithm, Variable, Condition, and Loop.
3. Draw the Scratch block for: (a) a "for" loop, (b) an "if" condition, (c) setting a variable.
4. What is the "Stage" and what is a "Sprite"?
5. Why is debugging important?

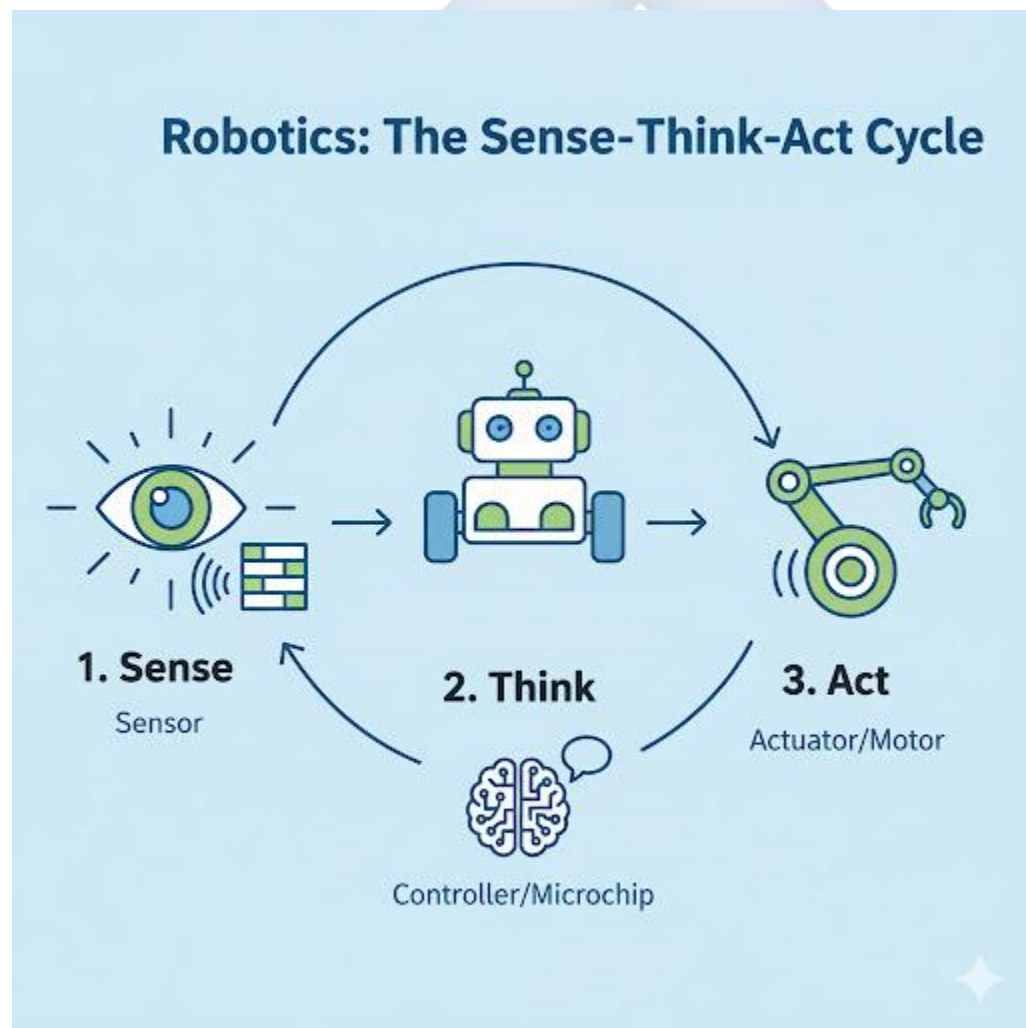
THIRD TERM – JSS 2 DIGITAL TECHNOLOGIES LESSON NOTES

WEEK 1: INTRODUCTION TO ROBOTICS

Topic: Introduction to Robotics

Breakdown (Subtopics):

- Meaning and types of robots
- Components of a robot
- Uses of robots



Performance Objectives: By the end of this lesson, students should be able to:

1. Define "Robot" and "Robotics."
2. List and describe the three main types of robots.
3. Identify the four main components common to most robots.
4. State at least four different uses of robots in various industries.

Lesson Content:

1. Meaning of Robot and Robotics

- **A. Robot:**
 - **Definition:** A **robot** is a programmable machine designed to automatically carry out a series of complex actions or tasks. It is a machine that can sense its environment, "think" (process data), and act on that environment.
 - **Key Idea:** A robot is different from a simple machine (like a toaster) because it is *programmable* and often *autonomous* (can operate on its own).
- **B. Robotics:**
 - **Definition:** **Robotics** is the branch of technology (a combination of computer science, mechanical engineering, and electrical engineering) that deals with the **design, construction, operation, and application of robots.**

2. Types of Robots Robots can be classified based on how they operate:

1. **Pre-Programmed Robots:**
 - These robots are programmed to do *one specific task* over and over again, in a controlled environment where nothing changes. They have no "awareness."
 - *Example:* A robotic arm on a car assembly line that does nothing but weld a car door in the exact same spot 800 times a day.
2. **Teleoperated Robots (Telebots):**
 - These robots are controlled *remotely* by a human operator. The human makes all the decisions.

- *Examples:* A bomb-disposal robot controlled by a soldier from a safe distance. A surgical robot (like the da Vinci) controlled by a surgeon. A deep-sea submersible.

3. **Autonomous Robots:**

- These robots can **sense their environment** and **make their own decisions** without direct human control. They often use Artificial Intelligence (AI).
- *Examples:* A **robot vacuum cleaner** (like a Roomba) that avoids furniture. A **self-driving car**. A **Mars rover** (like *Perseverance*) that navigates the Martian surface on its own.

3. **Basic Components of a Robot** Almost every robot has these four parts:

1. **Controller (The "Brain"):** This is the computer (microprocessor) that runs the software (the program). It processes information from the sensors and decides what to do.
2. **Sensors (The "Senses"):** These are the devices that gather information about the robot's environment.
 - *Examples:* Cameras (sight), microphones (sound), touch sensors, ultrasonic sensors (to measure distance, like a bat).
3. **Actuators (The "Muscles"):** These are the parts that make the robot *move* or *act*.
 - *Examples:* Electric motors, pistons, gears, wheels, grippers (hands).
4. **Power Supply (The "Food"):** This provides the energy for the robot to work.
 - *Examples:* Battery (for mobile robots), electrical cord (for stationary robots).

4. **Uses of Robots (Applications)** Robots are used, especially for tasks that are **D-D-D (Dull, Dirty, or Dangerous)** for humans.

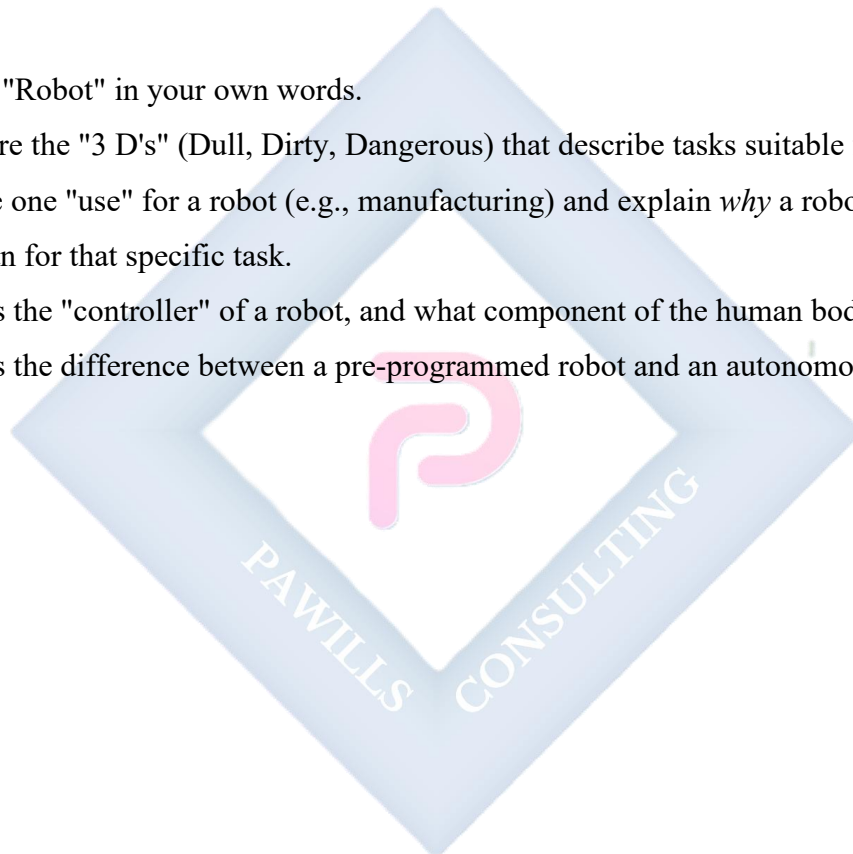
- **Manufacturing:** (Dull) Assembling cars, lifting heavy objects, painting.
- **Medicine:** (Precise) Performing complex, minimally invasive surgery.
- **Exploration:** (Dangerous) Exploring deep oceans, volcanoes, or other planets (like Mars) where humans cannot go.
- **Military & Security:** (Dangerous) Bomb disposal, surveillance drones.
- **Logistics:** Sorting packages in a warehouse (like Amazon).
- **Home:** (Dull) Robot vacuum cleaners, smart lawnmowers.

Evaluation:

1. What is the difference between a "robot" and "robotics"?
2. What is an "autonomous" robot? Give one example.
3. What is a "teleoperated" robot? Give one example.
4. List the four main components of a robot.
5. What are the "actuators" of a robot? What is their job?

Assignment:

1. Define "Robot" in your own words.
2. What are the "3 D's" (Dull, Dirty, Dangerous) that describe tasks suitable for robots?
3. Choose one "use" for a robot (e.g., manufacturing) and explain *why* a robot is better than a human for that specific task.
4. What is the "controller" of a robot, and what component of the human body is it like?
5. What is the difference between a pre-programmed robot and an autonomous robot?



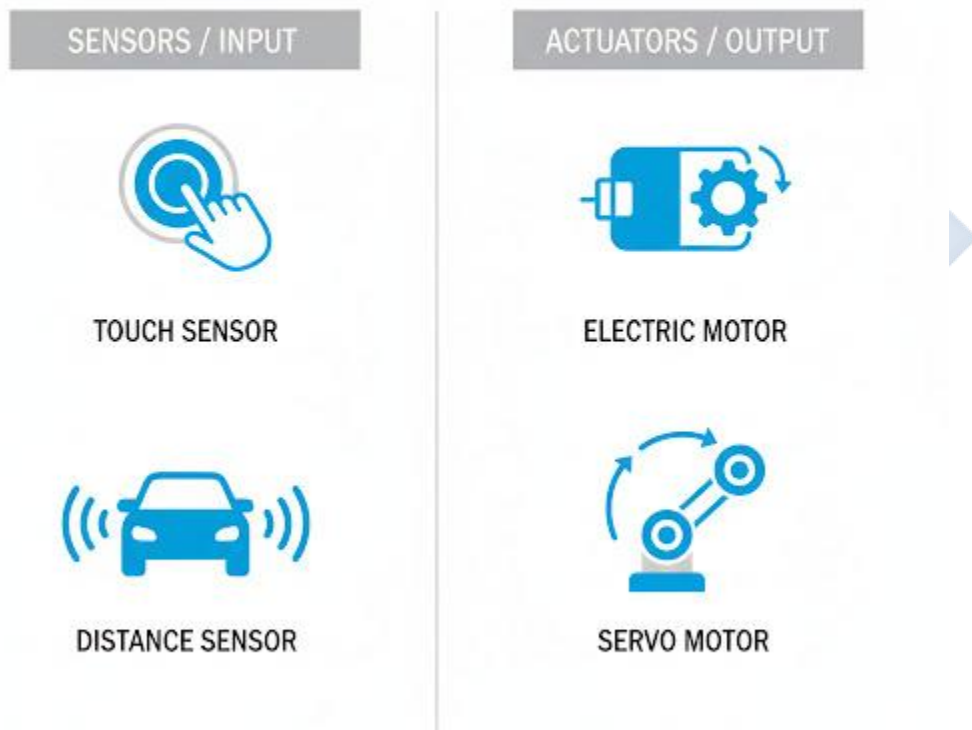
WEEK 2: PARTS AND FUNCTIONS OF A ROBOT

Topic: Parts and Functions of a Robot

Breakdown (Subtopics):

- Sensors, actuators, controllers
- Role of software in robotics

ROBOT INPUTS (SENSE) AND OUTPUTS (ACT)



Performance Objectives: By the end of this lesson, students should be able to:

1. Define and list three different types of sensors.
2. Define and list three different types of actuators.
3. Explain the role of the controller as the "brain."

4. Explain the relationship between hardware (parts) and software (program) in a robot.
5. Draw a simple diagram showing how sensors, controllers, and actuators work together.

Lesson Content:

1. The "Sense-Think-Act" Cycle A robot's operation can be described by a simple loop:

1. **SENSE:** The robot uses its **Sensors** to gather data about its environment.
2. **THINK:** The **Controller** (brain) processes this data using its **Software** (program) and makes a decision.
3. **ACT:** The Controller sends a command to the **Actuators** (muscles) to move or perform an action. ...This cycle repeats continuously.

2. Deep Dive: Sensors (The "Senses")

- **Definition:** Sensors are the hardware components that allow a robot to perceive (sense) its surroundings.
- **Types and Examples:**
 - **Light Sensors (Photocells):** Measure the amount of light.
 - *Use:* A robot that can follow a black line on a white floor.
 - **Touch Sensors (Bumpers):** A simple switch that is pressed when the robot hits an obstacle.
 - *Use:* A robot vacuum cleaner that "knows" it has hit a wall and needs to turn.
 - **Ultrasonic Sensors (Sonar):** Send out high-frequency sound waves (that humans can't hear) and measure how long it takes for the echo to bounce back.
 - *Use:* To measure the *distance* to an object, just like a bat. Used in self-driving cars.
 - **Infrared (IR) Sensors:** Can detect heat or be used to see in the dark.
 - *Use:* A rescue robot searching for a person in a dark, collapsed building.
 - **Cameras (Vision):** The "eyes" of the robot, allowing it to see and use AI to recognize objects or faces.

3. Deep Dive: Actuators (The "Muscles")

- **Definition:** Actuators are the components that convert energy (usually electrical) into physical motion. They are what make the robot *do* things.
- **Types and Examples:**
 - **Electric Motors:** The most common actuator. They *spin*.
 - *Use:* Turning wheels, spinning propellers on a drone, moving a robotic arm.
 - **Servos:** A special type of motor that can be controlled to move to a *precise position or angle*.
 - *Use:* Steering the front wheels of a robotic car, making a robot's "head" turn.
 - **Linear Actuators (Pistons):** Move in a straight line (push and pull) instead of spinning.
 - *Use:* Opening and closing a robotic gripper (hand).
 - **Gears:** Used with motors to increase power (torque) or speed.

4. Deep Dive: Controller and Software

- **Controller (The "Brain"):**
 - This is the hardware, usually a **microprocessor** or **microcontroller** (a small computer on a chip).
 - It runs the program and physically connects all the sensors and actuators.
 - *Example:* An Arduino board, a Raspberry Pi.
- **Software (The "Thoughts" / "Program"):**
 - This is the set of instructions (like the Scratch code you learned) that tells the controller *how* to "think."
 - The software *is* the intelligence. Without software, the controller is just a useless chip.
 - **The Logic:** The software is what contains all the IF...THEN statements (conditions) and LOOPS (Week 6 & 8, Term 2).
 - *Example Code (Pseudocode):*

1. START LOOP
2. Check Ultrasonic Sensor
3. IF distance < 10cm THEN:
4. Tell Motors to STOP
5. Tell Motors to TURN LEFT for 1 second
6. ELSE:
7. Tell Motors to GO FORWARD
8. END LOOP

Evaluation:

1. What are the three steps in the "Sense-Think-Act" cycle?
2. What is a sensor? Give two different examples.
3. What is an actuator? Give two different examples.
4. What is the "brain" of the robot called?
5. What is the "intelligence" or "thoughts" of the robot called?

Assignment:

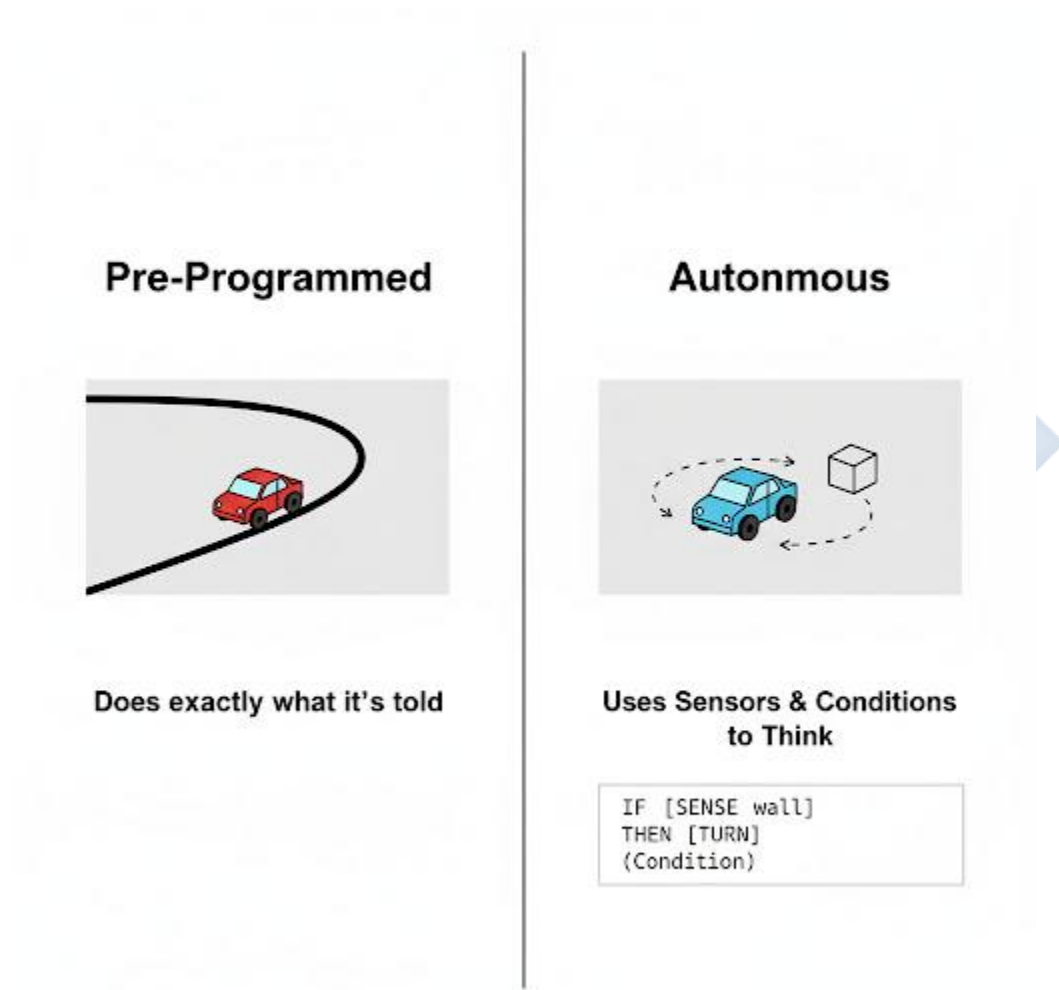
1. Define Sensor, Actuator, and Controller.
2. For each human sense, name a robot sensor that does the same job: (a) Eyes, (b) Ears, (c) Skin (touch).
3. What is the difference between a regular motor and a servo motor?
4. Draw a simple diagram showing the "Sense-Think-Act" cycle.
5. What is more important: a robot's hardware (body) or its software (thoughts)? Explain your answer.

WEEK 3: BASIC ROBOTIC MOVEMENTS

Topic: Basic Robotic Movements

Breakdown (Subtopics):

- Programming simple movements
- Concept of automation



Performance Objectives: By the end of this lesson, students should be able to:

1. Define "Automation."
2. Explain the link between robotics and automation.
3. Write pseudocode for a simple robotic movement (e.g., move in a square).

4. Describe how a pre-programmed robot (like a factory arm) works.
5. Explain how adding sensors makes a robot's movement "smarter."

Lesson Content:

1. Concept of Automation

- **Definition: Automation** is the use of technology (especially control systems, computers, and robots) to operate a process or system **automatically**, with minimal or no human intervention.
- **Goal:** To increase efficiency, reduce errors, save money, and perform tasks that are too boring or dangerous for humans.
- **Robots and Automation:** Robots are the *agents* of automation. They are the "hands" that perform the automated tasks.
 - *Example:* A fully automated car factory uses pre-programmed robots (Week 1) to weld, paint, and assemble cars 24/7 without human help. This is automation.

2. Programming Simple Movements (Pre-Programmed) This is the simplest form of robotics. The robot does not use sensors; it just follows a "blind" set of instructions. This is like using only the Motion blocks in Scratch.

- **Algorithm (To move in a square):**
 - This is an algorithm (Term 2, Week 2).
 - It uses a loop (Term 2, Week 8).
 - **Pseudocode:**
 1. START PROGRAM
 2. LOOP 4 TIMES:
 3. Tell Motors to MOVE FORWARD for 2 seconds
 4. Tell Motors to STOP
 5. Tell Right Motor to TURN FORWARD for 0.5 seconds (This makes the robot spin 90 degrees)
 6. Tell Right Motor to STOP
 7. END LOOP

8. END PROGRAM

- **Application:** This is perfect for a factory, where the robot's task never changes. It is *not* good for your home, where it would crash into furniture.

3. Programming "Smarter" Movements (Autonomous) To make a robot useful in the "real world" (an uncontrolled environment), its movements must *react* to its surroundings. This means we must combine Loops, Sensors, and Conditions (Term 2, Weeks 6 & 8).

- **Algorithm (To move forward until it hits a wall):**

- **Pseudocode:**

1. START PROGRAM
2. START LOOP (FOREVER):
3. Check Touch Sensor
4. IF Touch Sensor is PRESSED THEN:
5. Tell Motors to MOVE BACKWARD for 1 second
6. Tell Motors to TURN RIGHT for 1 second
7. ELSE:
8. Tell Motors to MOVE FORWARD
9. END IF
10. END LOOP

- **Result:** This simple program creates an "intelligent" movement. The robot will now wander around a room, and every time it hits a wall, it will back up and turn away. This is a basic autonomous robot.

Evaluation:

1. What is "automation"?
2. What is the main difference between a "pre-programmed" robot's movement and an "autonomous" robot's movement? (Hint: sensors).
3. What programming concept (e.g., variable, loop, condition) is used to *repeat* an action?
4. What programming concept is used to make a *decision* (e.g., "IF I hit a wall...")?
5. Write simple pseudocode (3 steps) to make a robot move forward, stop, and then turn left.

Assignment:

1. Define "automation." Give one example from real life.
2. Draw a flowchart for the "move in a square" algorithm from the lesson.
3. Write the *pseudocode* for an algorithm that makes a robot *stop* moving when its Light Sensor (Week 2) detects "darkness" (e.g., it runs on a black line and stops at the end).
4. How is this "line-following" robot (Question 3) more "intelligent" than the "move in a square" robot?
5. What is the role of programming (software) in making a robot move?



WEEK 4: ARTIFICIAL INTELLIGENCE (AI)

Topic: Artificial Intelligence (AI)

Breakdown (Subtopics):

- Meaning and types
- Applications in daily life



Performance Objectives: By the end of this lesson, students should be able to:

1. Define "Artificial Intelligence" (AI).
2. Differentiate AI from a simple pre-programmed robot.
3. Define "Machine Learning" as a type of AI.

4. Differentiate between Narrow AI (Weak AI) and General AI (Strong AI).
5. List at least five applications of AI they use in their daily lives.

Lesson Content:

1. Meaning of Artificial Intelligence (AI)

- **Definition: Artificial Intelligence (AI)** is a field of computer science focused on creating "intelligent" machines that can **think, learn, and perform tasks** that typically require human intelligence.
- **Human Intelligence:** This includes problem-solving, learning from experience, understanding language, and recognizing objects.
- **AI vs. Simple Program:**
 - **Simple Program:** A "dumb" follower. It *only* does what you programmed it to do. (e.g., The "move in a square" robot).
 - **AI Program:** A "learner." It *can change its own behaviour* based on new data or experiences. (e.g., A robot that "learns" the fastest path through a maze after 10 tries).

2. Machine Learning (ML)

- **Definition: Machine Learning** is the most common *type* of AI today. It is the process of "training" a computer to **learn from large amounts of data** and make predictions or decisions *without* being explicitly programmed for that task.
- **Analogy:**
 - **Traditional Programming:** You write the rules. IF email contains "prince" AND "money" THEN it is spam.
 - **Machine Learning:** You give the computer 1 million spam emails and 1 million good emails and say, "You figure out the rules." The AI *learns* the patterns itself.

3. Types of AI

- **A. Narrow AI (Weak AI):**

- **Definition:** This is the *only* type of AI we have today. It is an AI that is designed and trained to perform **one specific, narrow task**.
- **Characteristics:** It can be *very good* at that one task (e.g., better than a human at chess), but it has no consciousness, awareness, or ability to do anything else.
- **Examples:**
 - **Siri / Google Assistant:** Good at understanding your voice commands, but can't write a story.
 - **Google Search:** Excellent at finding web pages.
 - **Netflix Recommendations:** Good at guessing what movie you'll like.
 - **Spam Filter:** Good at catching bad emails.
- **B. General AI (Strong AI):**
 - **Definition:** This is a *hypothetical* (it does not exist yet) type of AI that would have the same level of intelligence as a human being.
 - **Characteristics:** It would be able. to learn, understand, and apply its intelligence to *any* problem, just like a person.
 - **Examples:** The intelligent, self-aware robots you see in movies (e.g., C-3PO in Star Wars, The Terminator).

4. Applications of AI in Daily Life AI is already all around you:

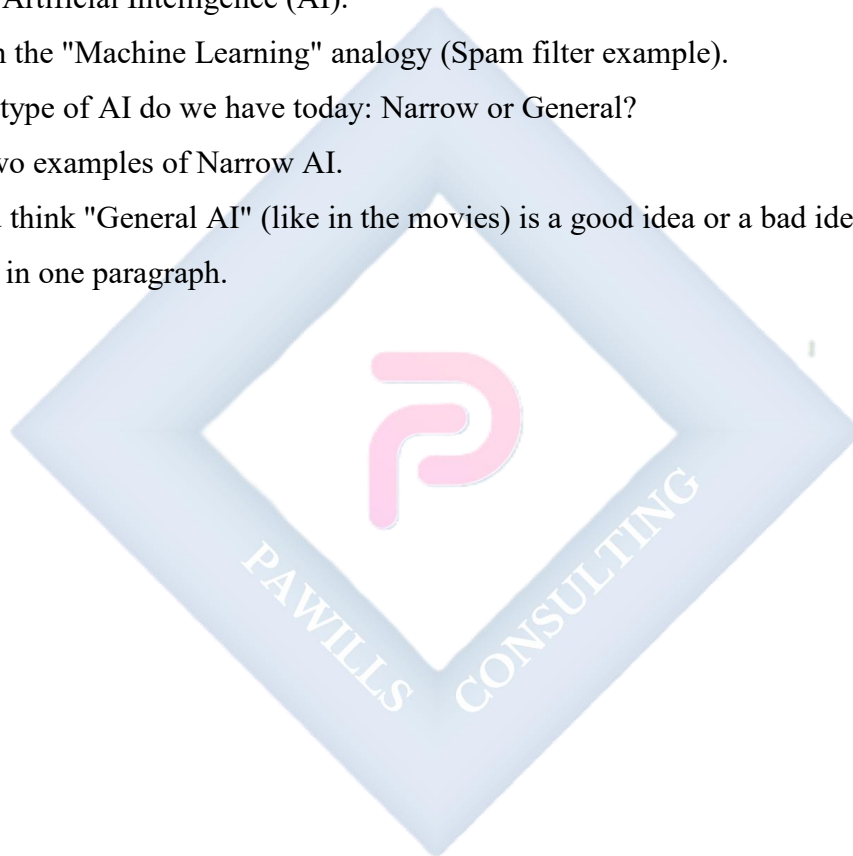
1. **Voice Assistants:** Siri, Google Assistant, Alexa.
2. **Recommendation Engines:** The "For You" page on **TikTok**, YouTube recommendations, Netflix suggestions. The AI learns what you like to watch.
3. **Search Engines:** Google Search uses AI to understand your query and rank the best results.
4. **Email Spam Filters:** Your email (e.g., Gmail) uses AI to automatically move spam to your junk folder.
5. **Digital Maps:** Google Maps uses AI to find the fastest route and predict traffic.
6. **Social Media:** AI identifies faces in photos (tagging) and decides what to show you in your feed.

Evaluation:

1. What is Artificial Intelligence?
2. What is "Machine Learning"?
3. What is the main difference between a simple program and an AI?
4. What is the difference between Narrow AI and General AI?
5. List three applications of AI that you (or your family) use every day.

Assignment:

1. Define Artificial Intelligence (AI).
2. Explain the "Machine Learning" analogy (Spam filter example).
3. Which type of AI do we have today: Narrow or General?
4. Give two examples of Narrow AI.
5. Do you think "General AI" (like in the movies) is a good idea or a bad idea? Explain your answer in one paragraph.

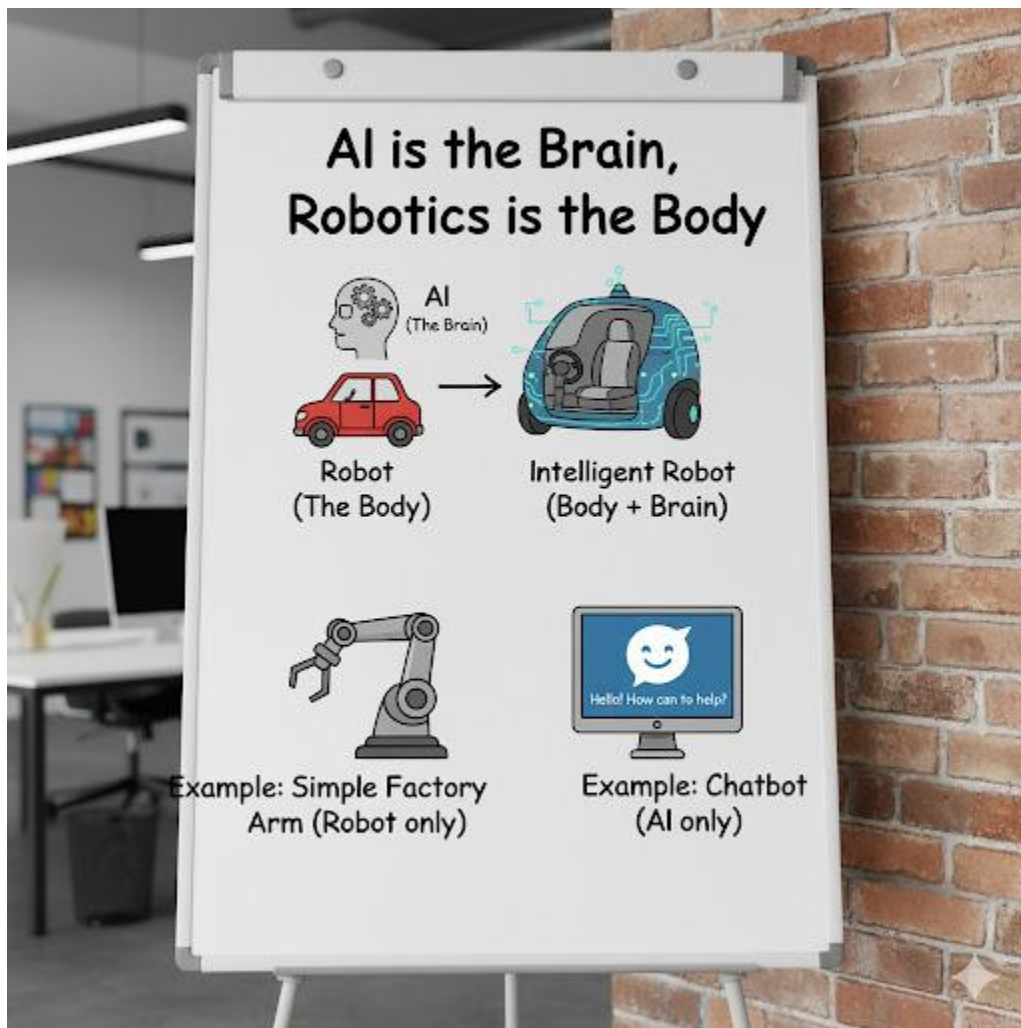


WEEK 6: DIFFERENCE BETWEEN AI AND ROBOTICS

Topic: Difference Between AI and Robotics

Breakdown (Subtopics):

- Interrelationship between AI and robotics
- Examples: drones, chatbots



Performance Objectives: By the end of this lesson, students should be able to:

1. Clearly state the difference between AI (the brain) and Robotics (the body).

2. Give an example of a robot *without* AI.
3. Give an example of an AI *without* a robot.
4. Give an example of an AI *inside* a robot.
5. Explain the function of a chatbot and a drone as examples.

Lesson Content:

1. AI vs. Robotics: The Key Difference This is a common point of confusion.

- **Robotics:** This is a field of *engineering* that deals with building and programming the **physical machine (the body)**. A robot is a physical thing that *moves* and *interacts* with the world.
- **Artificial Intelligence (AI):** This is a field of *computer science* that deals with creating the **intelligent software (the brain)**. An AI is a program that *thinks* and *learns*.

Analogy:

- A **Robot** is the **Car** (the hardware: wheels, engine, steering).
- **AI** is the **Driver** (the intelligence: sees the road, makes decisions).

2. The Four Possibilities You can have one without the other.

- **A. Robot WITHOUT AI (A "Dumb" Robot):**
 - **What it is:** A physical machine (robot) that runs on a *simple, pre-programmed* set of instructions (no AI).
 - **Example:** A car-factory arm that welds a door. It just repeats one motion. It has no "brain" and cannot make decisions.
- **B. AI WITHOUT a Robot (A "Disembodied" Brain):**
 - **What it is:** An intelligent program (AI) that exists only as software on a computer or server. It has no physical body to move.
 - **Example 1: A Chatbot.** A program (like ChatGPT or a bank's customer service bot) that uses AI to *understand* your text questions and *write* intelligent answers. It is pure AI, no robot.

- **Example 2: Google Search.** A massive AI that finds information, but it is not a physical machine you can touch.
- **C. AI + Robot (An "Intelligent" Robot):**
 - **What it is:** This is the "full package." It is an *intelligent brain (AI)* controlling a *physical body (robot)*. This is an autonomous robot.
 - **Example 1: A Drone.** A *drone* is the robot (body: propellers, camera). The *AI* (brain) allows it to "follow a person," "avoid trees," or "fly back home" on its own.
 - **Example 2: A Self-Driving Car.** The *car* is the robot (body). The *AI* is the brain that sees the road and controls the steering.
 - **Example 3: A Robot Vacuum.** The *vacuum* is the robot (body). The *AI* is the brain that maps the room and avoids furniture.
- **D. Neither AI nor Robot:**
 - *Example:* A simple calculator or Microsoft Word. It's just a simple program, not intelligent (AI) and not a moving machine (robot).

Evaluation:

1. What is the simple analogy for AI and Robotics? (Brain vs. Body).
2. Is a robotic arm in a car factory an example of AI? Why or why not?
3. Is a "Chatbot" a robot? Why or why not?
4. A self-driving car is an example of: (a) Just AI, (b) Just a Robot, (c) Both AI and a Robot.
5. What is a "drone"? Is it AI, a robot, or both?

Assignment:

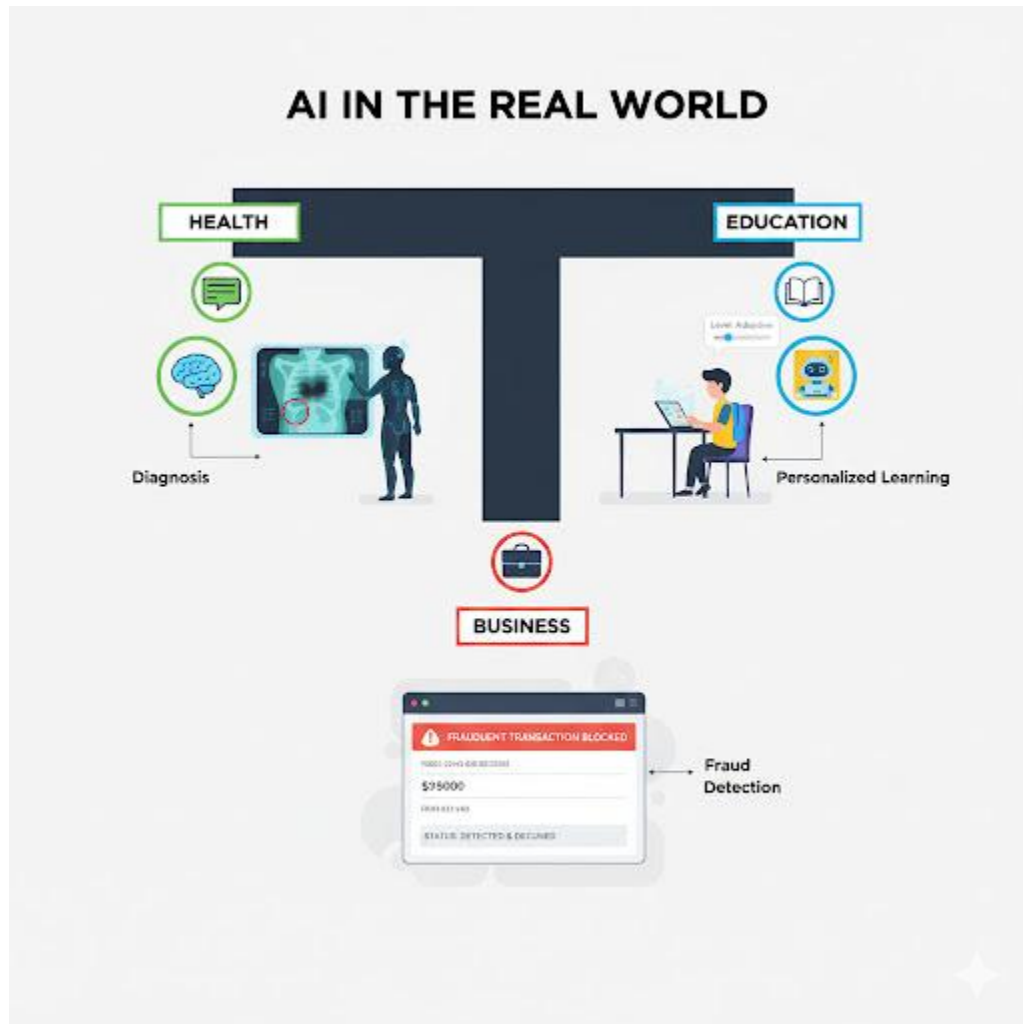
1. In a two-column table, list three differences between AI and Robotics.
2. Give one example of a robot *without* AI.
3. Give one example of an AI *without* a robot.
4. Give one example of an AI *inside* a robot.
5. What is a chatbot? How does it use AI?

WEEK 8: REAL-LIFE APPLICATION OF AI

Topic: Real-Life Application of AI

Breakdown (Subtopics):

- AI in education, health, and business



Performance Objectives: By the end of this lesson, students should be able to:

1. Describe one specific application of AI in healthcare.
2. Describe one specific application of AI in education.
3. Describe two specific applications of AI in business.
4. Discuss the benefits of using AI in these fields.

Lesson Content:

(This lesson expands on Week 4)

1. AI in Healthcare (Health-Tech)

- **A. Medical Diagnosis (AI "Doctors"):**
 - **Application:** AI programs are being trained to *read* medical images like **X-rays, CT scans, and MRIs**.
 - **Benefit:** In many cases, the AI can detect diseases (like cancer tumours or eye diseases) *earlier* and *more accurately* than a human doctor. It can also analyze thousands of scans in minutes.
- **B. Drug Discovery:**
 - **Application:** AI is used to simulate how different chemicals will react, speeding up the process of finding new medicines (drugs).
 - **Benefit:** It can take 10 years for humans to develop a new drug. AI can help cut this time down, saving lives faster.

2. AI in Education (Ed-Tech)

- **A. Personalized Learning:**
 - **Application:** AI-powered apps (like Duolingo for languages, or math apps) can *adapt* to your personal skill level.
 - **Benefit:** If you are finding a topic easy, the AI gives you harder questions. If you are struggling, it gives you more practice and hints. It's like having a personal tutor.
- **B. Administrative Tools for Teachers:**
 - **Application:** AI can help teachers by *automatically grading* multiple-choice tests or even simple essays.
 - **Benefit:** This frees up the teacher's time from (dull) marking so they can spend more time (creative) planning lessons and helping students.
- **C. Plagiarism Checkers:** AI (like Turnitin) reads a student's essay and compares it to billions of websites to see if they copied (plagiarized).

3. AI in Business and Finance

- **A. Customer Service (Chatbots):**

- **Application:** (Week 6). When you visit a bank's or an airline's website, a "chat" window pops up. This is usually an AI chatbot, not a human.
- **Benefit:** It can answer simple questions ("What time do you open?") 24/7, instantly, without needing to pay a human.

- **B. Fraud Detection (Banking):**

- **Application:** Your bank uses AI to *monitor* your account for strange activity. The AI learns your normal spending habits.
- **Benefit:** If your debit card is suddenly used in a different country (e.g., China) or for a huge, unusual amount, the AI will *instantly flag* it as fraud, block the transaction, and send you an alert.

- **C. E-Commerce (Online Shopping):**

- **Application:** When you shop on Jumia or Amazon, the "Customers who bought this also bought..." section is run by AI.
- **Benefit:** It recommends products you are more likely to buy, increasing sales for the business.

Evaluation:

1. How is AI used in healthcare to read X-rays?
2. What is "personalized learning" in education?
3. How do banks use AI for "fraud detection"?
4. What is a "chatbot"?
5. What is one benefit of using AI to help teachers grade papers?

Assignment:

1. Choose *one* field (Health, Education, or Business).
2. Write one paragraph (3-5 sentences) describing a specific use of AI in that field.
3. Write a second paragraph explaining the *benefits* of using AI for that task.
4. Find another example of AI in daily life (not from the lesson) and describe what it does.

5. What do you think is the *most* useful application of AI? Why?

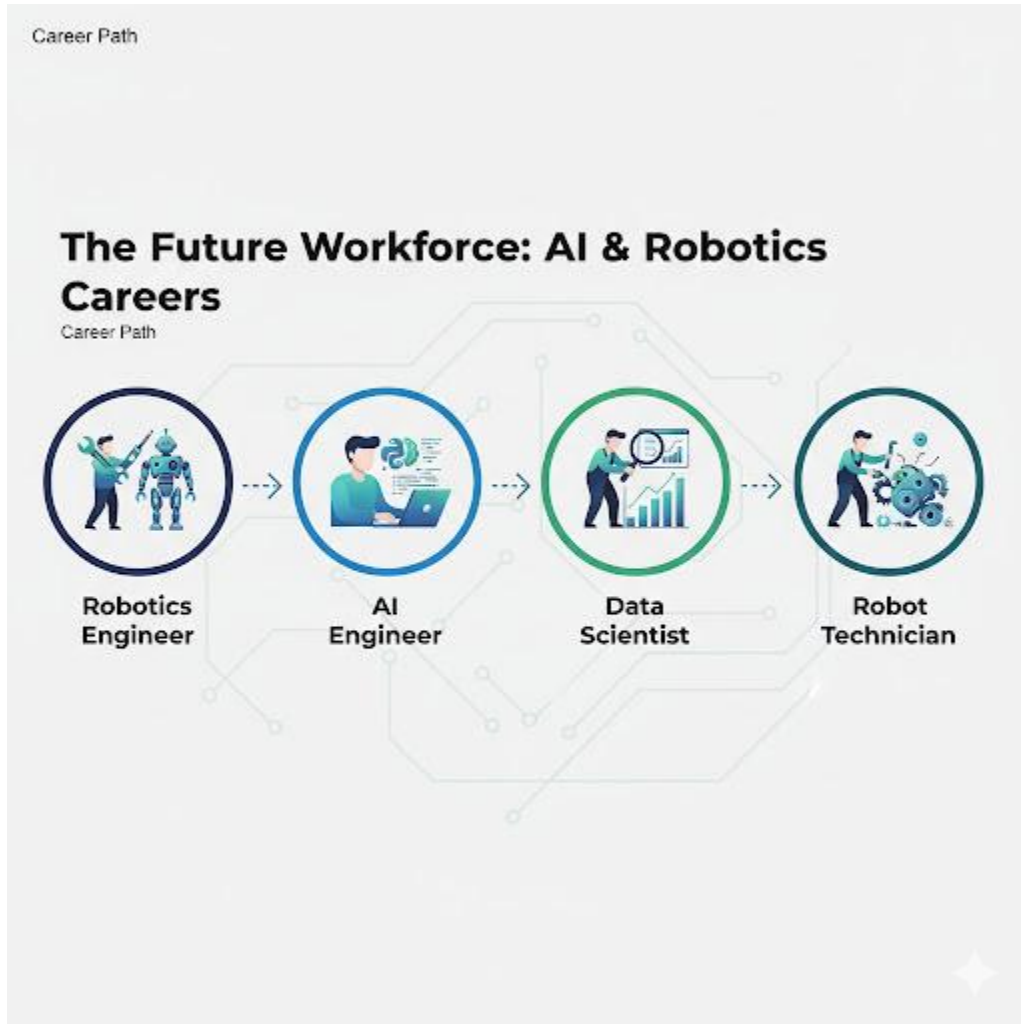


WEEK 9: CAREERS IN ROBOTICS AND AI

Topic: Careers in Robotics and AI

Breakdown (Subtopics):

- Job opportunities and skills needed



Performance Objectives: By the end of this lesson, students should be able to:

1. List at least four different career paths in AI and Robotics.
2. Describe the function of each career path.
3. Identify the key skills required for these careers (STEM, Programming).
4. Explain why these careers are important for the future.

Lesson Content:

1. Why These Careers are Important AI and Robotics are *future-proof* fields. They are not just growing; they are changing *every other* industry (like medicine, farming, and art). The jobs in this area are some of the most high-demand and high-paying in the world.

2. Job Opportunities (Careers)

- **A. Robotics Engineer:**

- **What they do:** The "builder." They design, build, and test the physical robot (the hardware).
- **Skills:** Mechanical Engineering, Electrical Engineering, understanding sensors and actuators.

- **B. AI / Machine Learning (ML) Engineer:**

- **What they do:** The "trainer." They design and build the AI *models* (the brain). They write the complex algorithms, gather the data, and "train" the AI to learn.
- **Skills:** Strong programming (especially **Python**), advanced mathematics, understanding data.

- **C. Data Scientist:**

- **What they do:** The "analyst." They collect, clean, and analyze huge amounts of data to find patterns and insights. Their findings are used to "feed" the AI.
- **Skills:** Statistics, data analysis, programming.

- **D. Robot Technician:**

- **What they do:** The "mechanic." They are a vocational/technological role (Term 2 Science, Week 10). They are responsible for *installing, maintaining, and repairing* robots in places like factories or hospitals.
- **Skills:** Practical skills, electronics, troubleshooting.

- **E. AI Ethicist / AI Safety Researcher:**

- **What they do:** The "philosopher/guide." They study the *dangers* and *biases* of AI. They try to answer hard questions like, "What moral rules should we program into a self-driving car?"
- **Skills:** Philosophy, ethics, law, computer science.

3. Skills Needed for a Future in AI/Robotics To work in this field, you need a strong foundation in:

1. STEM:

- Science (especially Physics)
- Technology (Digital Technologies)
- Engineering (Mechanical, Electrical)
- Mathematics (This is *very* important for AI)

2. Programming: This is the language of AI. **Python** is the most popular language for AI and robotics today.

3. Problem-Solving: (Term 2, Week 2). The ability to break down complex problems.

4. Creativity: The ability to think of new, innovative ideas.

Evaluation:

1. What is the main job of a Robotics Engineer?
2. What is the main job of an AI/Machine Learning Engineer?
3. What is the main job of a Robot Technician?
4. What does "STEM" stand for?
5. What is the most popular programming language for AI?

Assignment:

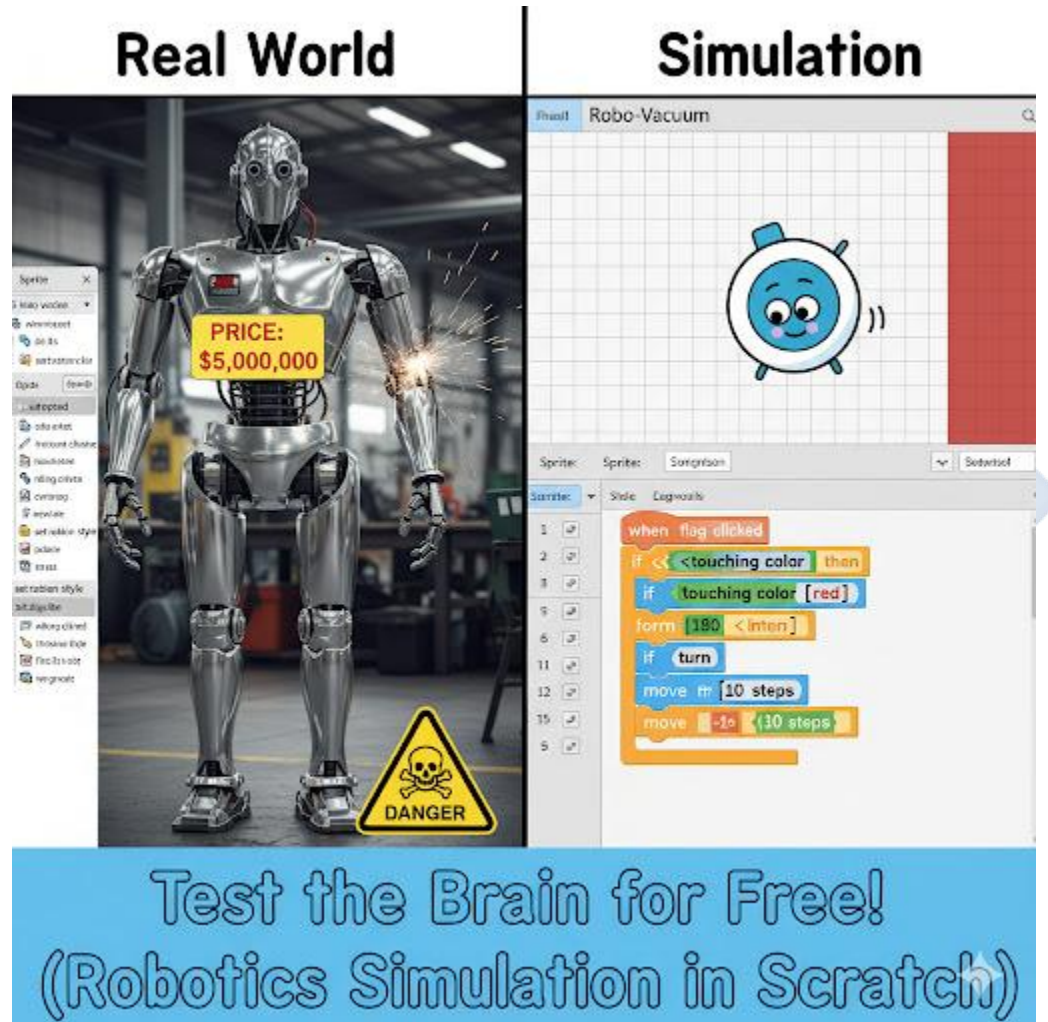
1. List four different careers in AI and Robotics.
2. Choose *one* of those careers and describe what a person in that job does all day.
3. What is the difference between a Robotics Engineer (who builds the body) and an AI Engineer (who builds the brain)?
4. What subjects in school are most important if you want to have a career in AI? (List at least 3).
5. Are you interested in a career in AI or Robotics? Why or why not?

WEEK 10: CLASS PROJECT: BUILDING A SIMPLE ROBOT (SIMULATION)

Topic: Class Project: Building a Simple Robot (Simulation)

Breakdown (Subtopics):

- Hands-on digital model or simulation



Performance Objectives: By the end of this lesson, students should be able to:

1. Define "Simulation."
2. Explain why simulation is used in robotics.

3. Apply their knowledge of loops and conditions to design a simple robot program.
4. Build a simple "robot simulation" in Scratch.
5. Debug their simulation to make it work as intended.

Lesson Content:

1. What is a Simulation?

- **Definition:** A **simulation** is a *digital model* (a program) that imitates the behavior of a real-world system or process.
- **Why use it?** Building a *physical* robot is expensive (you need motors, sensors, batteries, etc.). A simulation allows us to design, program, and *test* our robot's "brain" (the software) on a computer *for free* before we build the physical body.

2. Our Project: "Robo-Vacuum" Simulation (in Scratch)

- **Goal:** To program a Sprite to move around the Stage like an autonomous robot vacuum, bouncing off the walls.
- **Assets:**
 - **Sprite 1:** The "Robot" (e.g., a simple circle or the "Gobo" sprite).
 - **Stage:** A background, perhaps with a "wall" drawn on it in a different colour (e.g., black).

3. The Algorithm (Sense-Think-Act): We need our robot to do two things at the same time:

1. *Movement:* Constantly move forward.
2. *Sensing:* Constantly check *if* it is touching a wall. If it is, it must back up, turn, and start moving again.

4. The Scripts (This uses ALL our knowledge from Term 2) (In the "Robot" Sprite's Script Area)

- **Script 1: The "Move" Engine (A Loop)**
 - This script just makes the robot go forward forever.

2. When green flag clicked (Events)
3. forever (Control)
4. { move (5) steps (Motion) }

- **Script 2: The "Sensor" Brain (A Loop with a Condition)**

- This script *independently* checks for the wall.

1. When green flag clicked (Events)
2. forever (Control)
3. { if <touching [edge]?> OR <touching color [#]?> (Sensing/Operators)
 - (Use the 'touching color' block and click the black colour of your wall)
4. then
5. { move (-50) steps (Motion) - *This makes it back up*
6. turn (clockwise) (90) degrees (Motion) - *This makes it turn*
7. wait (0.5) seconds (Control) } }

- **Running the Simulation:**

- When you click the green flag, *both* scripts run at the same time. The robot will move forward (Script 1) until its "sensor" (Script 2) detects a wall. When it does, Script 2 will *interrupt* the movement, make the robot back up and turn, and then Script 1 will take over again, moving it in the new direction.
 - **This is a simulation of an autonomous robot.**

Evaluation:

1. What is a "simulation"?
2. Why do engineers use simulations before building a real robot?
3. In our simulation, what block acts as the "motor" (Actuator)?
4. In our simulation, what block acts as the "sensor"?
5. What programming concept (loop, condition, variable) did we use to check for the wall?

Assignment:

1. Define "simulation."
2. Write the *pseudocode* for Script 2 (the "Sensor" brain) of our Robo-Vacuum.
3. **Debug:** What would happen if we *forgot* to add the move (-50) steps (back up) block?
(The robot would get stuck in a "jitter" loop, turning forever at the wall).
4. **Challenge (Write Pseudocode):** How would you add a "Dirt" sprite? (e.g., A *third* script that says: forever { if <touching [Dirt]?> then { play sound "pop", tell "Dirt" to hide } }).
5. Build the "Robo-Vacuum" simulation in Scratch. (This is a practical assignment).



WEEK 11: REVISION

Topic: Revision

Breakdown (Subtopics):

- Review of all topics

Performance Objectives: By the end of this lesson, students should be able to:

1. Define the key concepts of the term (Robotics, AI, Automation).
2. Explain the "Sense-Think-Act" model.
3. Discuss the relationship between AI and Robotics.
4. Review the careers and real-world applications of AI.

Lesson Content:

1. Summary of Term 3 Topics (The "AI & Robotics" Term):

- **Week 1 (Robotics):** A Robot is a programmable machine. Robotics is the study of robots.
Types: Pre-programmed, Teleoperated, Autonomous.
- **Week 2 (Parts):** Robots use the **Sense-Think-Act** cycle.
 - Sensors (sight, touch, distance) gather data.
 - Controller (brain) runs the software.
 - Actuators (motors, wheels) make it move.
- **Week 3 (Movement):** Automation is using robots to do tasks automatically. We use loops and conditions (from Term 2) to program all robot movements.
- **Week 4 (AI):** AI is the "brain" software that can *learn* and *make decisions*.
 - Narrow AI (what we have today) is good at one task (e.g., Google Search, Siri, TikTok "For You" page).
 - General AI (human-level) is still science fiction.
- **Week 6 (AI vs. Robot):** A Robot is the *body*. AI is the *brain*.
 - You can have AI without a robot (e.g., a Chatbot).
 - You can have a robot without AI (e.g., a factory arm).

- An *intelligent* robot is AI + Robot (e.g., a Drone, a self-driving car).
- **Week 8 (Applications):** AI is used in Health (finding cancer), Education (personalized learning), and Business (fraud detection).
- **Week 9 (Careers):** Jobs include Robotics Engineer (builds body), AI Engineer (builds brain), and Data Scientist (analyzes data).
- **Week 10 (Project):** We built a Simulation of a robot in Scratch to test our software.

2. Class Activity: The Big Picture Discussion

- How are the concepts from Term 2 (Variables, Loops, Conditions) the *essential* tools needed to build the software for a robot (Term 3)?
- Is an autonomous robot (like a self-driving car) an example of Narrow AI or General AI? Why?
- What is the most useful application of AI you learned about? What is the most dangerous?

Evaluation: This week's evaluation is the Third Term Examination.

Assignment:

1. Study all notes from Week 1 to Week 10 for the examination.
2. Draw the "Sense-Think-Act" diagram.
3. Explain the difference between AI and Robotics using the "brain/body" or "driver/car" analogy.
4. List two examples of Narrow AI.
5. List two careers in this field and what they do.